알바급여 정석 - 노무 계산 로직 및 전자서류 총망라 명세서

> ⚠️ ***[대외비 · CONFIDENTIAL]***
>
> *본 문서는 '알바급여 정석(Alba Pay)' 시스템의 **핵심 계산 엔진 로직, 내부 변수명, 보안 메커니즘, 및 영업 비밀(Trade Secret)에 해당하는 기술 설계 정보**를 포함하고 있습니다.*
>
> - ***배포 범위**: 본 문서는 노무사 자문, 내부 QA, 또는 투자 실사(Due Diligence) 목적으로만 제공되며, 수신자 외 **제3자에게 무단 배포·복제·전달을 엄격히 금지**합니다.*
> - ***전자 보관 시**: 본 문서의 전자 사본은 **암호화된 저장 매체**에 보관하고, 열람 후 불필요한 사본은 즉시 파기하여 주시기 바랍니다.*
> - ***법적 고지**: 본 문서에 포함된 계산 알고리즘, 변수명, 데이터 구조는 **부정경쟁방지 및 영업비밀보호에 관한 법률**에 의해 보호받는 영업 비밀에 해당하며, 무단 유출 시 민·형사상 책임을 질 수 있습니다.*
> - ***문서 관리 번호**: `ALBAPAY-TECH-2026-001` | **보안 등급**: 대외비(Confidential) | **최종 갱신**: 2026-05-10*

> *본 문서는 '알바급여 정석(Alba Pay)' 시스템 내에 구현된 **주요 계산 로직, 내부 변수명, 예외 처리(Edge Cases) 및 방어 로직을 정리하여 노무사 제출용**으로 작성된 기술 명세서입니다. 코드를 몰라도 이해할 수 있도록 쉽게 풀어 작성되었습니다.*
>
> *※ 본 문서에서 설명하는 시스템 기능은 노무 관련 리스크를 줄이기 위한 보조 도구로서의 역할을 하며, 최종적인 법률 해석과 판단은 노무사 또는 전문가와의 상담을 통해 확인하시기 바랍니다.*

---

1. 상시근로자 수 산정 및 5인 이상/미만 분기 로직

상시근로자수는 근로계약 체결 및 수당(야간/연장/휴일) 발생 가산율을 결정하는 가장 핵심적인 지표입니다.

1.1. 원칙적 산정 로직 및 2분의 1 예외 규정 (근로기준법 시행령 제7조의2)

- **핵심 판별 변수**: `isFiveOrMore` (Boolean)
- **산정 기간**: **Rolling 30일** (오늘 기준 과거 30일) — 근로기준법 시행령 제7조의2 "법 적용 사유 발생일 전 1개월" 준수 [UPDATE 2026-05-14]
- **시스템 로직**:
  1. **Rolling 30일 기간** 내 출근한 전체 근로자 수(`totalPersonDays`)를 해당 기간의 영업일수(`operatingDays`)로 나눈 평균값을 산출합니다.
  2. **1/2 예외 규정 방어 로직**:
     - 평균 5인 이상이라도, 5인 미만으로 근무한 가동일수가 절반(1/2) 이상이면 👉 **5인 미만 사업장으로 강제 하향 (`isFiveOrMore = false`)**.
     - 평균 5인 미만이라도, 5인 이상으로 근무한 가동일수가 절반 이상이면 👉 **5인 이상 사업장으로 강제 상향 (`isFiveOrMore = true`)**.

1.2. 상시근로자 5인 이상/미만 전환(변동) 처리 로직

사업장의 규모 변화(채용, 퇴사 등)로 인해 상시근로자 수가 5인 이상에서 5인 미만으로, 혹은 그 반대로 변동될 경우 알바페이 시스템의 자동 대응 로직입니다.

1.2.1. 급여 정산 기간(Payroll Period) 단위의 독립적 상태 고정 (Snapshot)

- **정산 확정 시점의 상태 고정**: 사장님이 매월 급여를 계산/확정하는 시점에 "이번 정산 기간 동안 상시근로자 5인 이상이었습니까?"라는 확인을 거치며, 해당 월의 급여 명세서에는 확정된 `isFiveOrMore` 값이 스냅샷으로 영구 기록됩니다.
- **과거 급여의 무결성 보장**: 현재 매장이 5인 미만으로 축소되었더라도, 과거 5인 이상 시절에 확정된 급여 명세서(가산수당 1.5배 지급 내역 등)는 현재 매장 설정의 영향을 받지 않고 합법적으로 보존됩니다.

1.2.2. 수당 및 연차 처리의 동적 변환

- **CASE 1: 5인 이상 → 5인 미만으로 전환 시 (축소)**
  - **가산수당 (연장/휴일/야간)**: 전환된 시점 이후의 정산 월부터는 가산수당 배율이 1.5배에서 1.0배(기본시급)로 자동 하향 적용됩니다.
  - **연차 보존 정책 (Plan A) [UPDATE 2026-05-13]**: 5인 미만으로 전환된 달부터는 신규 연차 발생(Accrual)이 중단됩니다. 단, 5인 이상 시절에 이미 부여된 연차 이력은 '최소 보장 연차(Guaranteed Leave)'로 고정되어 소멸되지 않으며, 잔여 연차가 마이너스로 떨어지지 않고 안전하게 보존되어 노무 분쟁을 예방합니다.
- **CASE 2: 5인 미만 → 5인 이상으로 전환 시 (확장)**
  - **가산수당**: 전환된 정산 월부터 즉시 연장/야간/휴일 근무에 대해 1.5배의 가산수당 로직이 자동 적용됩니다.

- **연차**: 전환된 시점부터 1년 미만자의 경우 1개월 만근 시 1개의 연차가 새롭게 발생하기 시작합니다.

1.2.3. 수동 설정 우선순위(Override) 및 분쟁 예방 스냅샷 (Manual Override & Compliance Snapshot) [UPDATE 2026-05-13]

- **수동 설정 강제 적용 (Override)**: 알바페이 엔진의 자동 인원 추정 로직을 사장님이 '수동 설정(Manual)'으로 무시(Override)할 수 있습니다. 수동으로 `manual_5plus` 또는 `manual_under5` 를 설정하면, 자동 계산 결과와 관계없이 사장님의 설정이 최우선 적용됩니다. UI 갱신 시에도 이 수동 설정값이 자동으로 덮어씌워지지 않도록 방어 로직(`_finalizeIsFiveOrMoreBeforePayroll`)이 견고하게 적용되어 있습니다.
- **분쟁 예방 목적의 사유 스냅샷 보존**: 사장님이 수동으로 5인 이상/미만을 강제 전환할 때 입력한 '전환 사유(Reason)'는 알바생에게 노출되지 않으며, 오직 급여 명세서 스냅샷(`LaborDocument`) 데이터베이스에만 영구 보존됩니다. 이는 추후 노무 분쟁 발생 시 사장님을 보호하기 위한 확실한 근거 자료(Audit Trail)로 활용됩니다.

1.2.4. 자동 산출(Automatic) 로직의 근태 기반 및 데이터 정합성 체계 [UPDATE 2026-05-14]

- **근태(Attendance) 기반 1순위 산출**: 자동 산출 모드(Automatic)에서는 기본적으로 직원들의 실제 '출퇴근 기록(Attendance)'을 기반으로 가동일수와 출근 인원을 계산하여 5인 이상 여부를 판별합니다.
- **근태(Attendance) 100% 기반 산출 체계 강화**: 기존에 존재하던 '계약 보조값(Contract Fallback)' 추정 로직은, 알바생의 근무 교대나 대타 투입 시 본래 근무자와 대타 근무자가 이중으로 카운트되어 상시근로자 수가 과다 산출되는 치명적인 부작용을 일으켰습니다. 이를 해결하기 위해 보조 추정 로직을 전면 폐기하였으며, 현재는 오직 **직원들이 실제로 찍은 '출퇴근 바코드 기록(Attendance)'만을 100% 신뢰하여 상시근로자를 산출**하도록 방어 로직을 강화했습니다.
- **clockIn 날짜만 카운트 (clockOut 유령 카운트 방지) [UPDATE 2026-05-14]**: 직원이 퇴근을 당일에 찍지 않고 다음 날 새벽에 찍는 경우, clockOut 날짜까지 카운트하면 실제로 출근하지 않은 날에 유령 인원이 추가되어 5인 이상 일수가 부풀려지는 버그가 발생했습니다. 이를 해결하기 위해 **출근(clockIn) 날짜만 기준으로 해당 날의 근무 인원을 산정**하며, clockOut 날짜는 카운트에서 완전히 제외합니다.
- **파견직(Dispatch) 제외 [UPDATE 2026-05-14]**: 파견직 근로자(`workerType == 'dispatch'`)는 용역업체 소속이므로 매장의 상시근로자 수 산정에서 제외됩니다. `WorkerService.getAll()` 에서 파견직을 필터링하여 `activeStaffIds` 에 포함하지 않습니다.
- **삭제된 직원의 고아 기록(Orphaned Attendance) 필터링 [UPDATE 2026-05-14]**: 테스트 등의 목적으로 등록 후 삭제된 직원의 출퇴근 기록이 Firestore에 잔존하는 경우, 해당 기록이 상시근로자 카운트에 포함되어 인원수가 부풀려지는 문제가 발생했습니다. 현재 등록된 활성 직원의 ID 집합(`activeStaffIds`)과 대조하여, **등록되지 않은 staffId의 attendance 기록은 카운트에서 자동 제외**합니다.

1.3. 일시적 대타 근무로 인한 5인 이상 초과 시 방어 로직 (Edge Case)

- **상황 (Scenario)**: 평소 5인 미만 사업장인데, 정산기간(1일~31일) 중 휴가철이나 행사일 등에 대타 근무자가 일시적으로 투입되어 특정 며칠만 하루 출근 인원이 5명을 넘긴 경우.
- **노무적 쟁점**: 하루라도 5인을 넘겼다고 해서 해당 정산 월 전체를 5인 이상 사업장으로 간주하고, 모든 직원에게 연장/야간 1.5배 가산수당을 줘야 하는가?
- **시스템 자동 방어 로직**:
  - 알바페이 엔진은 단순히 그 달의 최대 인원이나 일회성 이벤트를 보지 않고, 1.1항의 **'1/2 예외 규정(근로기준법 시행령 제7조의2)'**을 1일 단위로 정밀 추적하여 적용합니다.
  - 정산기간 31일의 영업일 중 대타가 투입되어 5인 이상이 된 날이 단 며칠(예: 3일)에 불과하다면, 나머지 28일은 5인 미만으로 가동된 것입니다.
  - 5인 미만 가동일수(28일)가 전체 영업일수(31일)의 절반(1/2)을 초과하므로, 시스템은 해당 월 전체를 **최종적으로 '5인 미만 사업장'으로 확정(Lock)**합니다.
  - **결과**: 사장님은 일시적인 대타 기용이나 스케줄 겹침 현상 때문에 예기치 않게 1.5배 가산수당 폭탄을 맞는 법적 위험(Legal Risk)을 크게 줄일 수 있습니다.

1.4. Rolling 30일 기반 연속 판정 및 대시보드·리포트 통일 [UPDATE 2026-05-14]

- **Rolling 30일 통일**: 대시보드(메인 화면)와 급여 리포트 모두 동일한 **Rolling 30일**(오늘 기준 과거 30일) 기간으로 상시근로자 수를 산정합니다. 이전에는 대시보드는 Rolling, 급여 리포트는 정산기간(Settlement Period)으로 각각 다른 기간을 사용하여 두 화면의 결과가 불일치하는 문제가 있었습니다.
- **연속 판정**: Rolling 30일 방식은 매일 산정 기간이 하루씩 이동하므로, 5인 이상 상태가 특정 월에 고정(Lock)되지 않고 **실시간으로 변동**됩니다. 오늘 5인 이상이었더라도 내일 직원이 줄면 즉시 5인 미만으로 전환됩니다.
- **기술적 구현**:
  - `standing_change_alert.dart` : Rolling 30일로 계산 → Firestore `stores/{storeId}` 문서에 `averageWorkers` , `daysWithFiveOrMore` , `totalBusinessDays` 기록
  - `payroll_dashboard_screen.dart` : 급여 리포트에서도 동일한 Rolling 30일로 독립 계산하여 표시
  - 두 경로 모두 동일한 `calculateStandingFromAttendances()` 함수를 사용하므로 결과가 항상 일치합니다.

1.5. 실시간 상태 전환 지점 및 앱 내 경고 시스템 (위험 순간)

- **전환 지점(위험 순간)의 정확한 정의**:

    - 알바생이 늘어나서 법적 상태가 '5인 미만'에서 '5인 이상'으로 뒤집히는 위험한 순간은, 수학적으로 **Rolling 30일 기간 내 '5인 이상 출근한 날'이 가동일수 중 절반(1/2)에 도달하거나 초과하게 되는 바로 그날의 출근 시점**입니다.
    - **예시 (30일 영업 기준)**:
        - **[안전 지대]**: 14일 동안 대타를 불러 하루 5인 이상이 근무했어도 절반(15일)을 넘지 않았으므로, 14일째까지는 아무리 직원이 많아 보여도 시스템은 법적으로 '5인 미만 사업장' 방어막을 유지합니다.
        - **[위험 순간]**: 하루 더 대타가 출근하여 5인 이상 근무한 날이 **15일째(딱 절반)가 되는 바로 그 출근 순간**, 평균 인원에 관계없이 해당 Rolling 30일 전체가 법적으로 '5인 이상 사업장'으로 확정 전환됩니다.

- **앱 내 실시간 경고 시스템 (`standing_change_alert.dart`)**:
    - 앱 대시보드 진입 시 `StandingChangeAlert` 위젯이 자동 실행되어 Rolling 30일 기반으로 상시근로자를 재계산합니다.
    - 이전 기간 대비 5인 이상/미만 상태가 전환(Flip)되면 즉각 사장님 앱 대시보드 하단에 **"⚠️ 5인 이상 전환 — 근로계약서 재작성 필요"**라는 주황색 경고 배너를 띄웁니다.
    - 변동 이력은 Firestore `stores/{storeId}/standingMeta/current` 문서에 기록되며, 이전 기간의 판정값과 비교하여 상태 전환 여부를 판별합니다.
    - **Firestore 보안 규칙 [UPDATE 2026-05-14]**: `standingMeta` 서브컬렉션은 매장 소유자(`isStoreOwner`)만 읽기/쓰기 가능하도록 보안 규칙이 설정되어 있습니다. 이 규칙이 누락될 경우 `PERMISSION_DENIED` 에러로 인해 대시보드 갱신이 차단될 수 있으므로, `firestore.rules` 에 반드시 해당 match 블록이 포함되어야 합니다.
    - 이를 통해 1.5배로 수당이 뛰는 월급제 직원의 계약서를 즉시 다시 쓸 수 있도록 안내하여, 노무 리스크를 최소화할 수 있도록 지원합니다.

1.6. 가족(동거하는 친족) 직원의 상시근로자 산정 처리

- **노무적 쟁점 (사용종속관계 여부)**:
    - 동거하는 친족(가족)이 매장에서 일할 때, 상시근로자 수에 포함되는지 여부의 핵심 쟁점은 **'사용종속관계(지휘·감독을 받으며 임금을 목적으로 근로를 제공하는지)'**의 존재 여부입니다.
    - 가족이더라도 일반 알바생들과 동일하게 정해진 출퇴근 시간을 지키고, 사장님의 업무 지시를 받으며 일정한 시급/월급을 받는다면 법적으로 '근로자'로 인정되어 **상시근로자 5인 산정 시 머릿수에 포함**되어야 합니다.
    - 반대로, 출퇴근이 자유롭고 사업 운영을 공동으로 하거나 단순히 일시적으로 돕는 수준이라면 근로자성이 부인되어 산정에서 제외됩니다.
- **앱에서의 유연한 대처 방법 (실무 가이드)**:
    - 알바페이의 엔진은 출석부(출퇴근 바코드)에 기록을 남긴 모든 인원을 근로자로 간주하여 1명으로 카운트합니다. 따라서 가족 직원의 실질적인 '사용종속관계' 여부에 따라 사장님이 아래와 같이 앱 사용 방식을 선택해야 합니다.
    - **[가족이 근로자가 아닌 경우 (도와주는 경우)]**: 가족 직원은 앱에 등록만 해두고, 매일 '출퇴근 바코드'는 찍지 않도록 안내합니다. 엔진은 출석 기록이 없는 직원을 카운트에서 자동 제외하므로, 상시근로자 수에 포함되지 않으면서 송금용 가계부 명단으로만 활용할 수 있습니다.
    - **[가족이 명확한 근로자인 경우 (또는 수동 통제 필요 시)]**: 가족의 근태 기록까지 앱으로 엄격히 관리해야 한다면 바코드를 찍게 하되, 만약 이로 인해 의도치 않게 5인 이상으로 판정되는 것을 막고 싶다면 사장님 앱 매장 설정에서 '상시근로자 산정 방식'을 **'수동 설정(Manual)'**으로 변경합니다. 이를 통해 사장님이 직접 5인 미만 상태로 고정(Lock)하여 가산수당 로직을 통제할 수 있습니다.

---

2. 출퇴근 기록 및 휴게시간 차감 로직 (노무 쟁점 방어)

2.1. 출퇴근 보정 및 지각 유예 (Tardiness Protection & Unauthorized Overtime Block)

- **관련 변수**: `clockIn` , `clockOut` , `graceMinutes` , `overtimeApproved`
- **구현 내용**:
    - **조기 출근 방어 (일찍 온 경우)**: 알바생이 스케줄(오전 9시)보다 일찍 와서 오전 8시 40분에 앱으로 출근을 찍었더라도, 사장님이 연장근로를 별도 승인하지 않는 이상 급여 계산용 시작 시간은 자동으로 원래 스케줄인 오전 9시로 보정(`payrollEffectiveClockIn`)됩니다.
    - **지연 퇴근 보정 (늦게 간 경우 - 무단 연장 리스크 감소)**: 근로자가 퇴근 시간(오후 6시)이 지났음에도 매장에 머물다가 오후 6시 20분에 늦게 퇴근 바코드를 찍은 경우입니다. 이때 사장님이 '연장근로 승인(`overtimeApproved = true`)'을 해주지 않으면, **급여 정산용 퇴근 시간은 원래 스케줄인 오후 6시로 자동 절삭(Cap)되어 보정**됩니다. 이를 통해 이른바 '수당 빼먹기(무단 체류에 의한 가짜 연장수당)' 리스크를 줄일 수 있습니다.
    - **지각 유예 (`graceMinutes`)**: 지각 유예 시간을 5분으로 설정한 경우, 9시 3분에 출근해도 지각 시급 삭감 없이 9시 정각 출근으로 인정해 주는 노무 편의 기능도 함께 동작합니다.

2.2. 출퇴근 기록 누락 시 예외 처리 (Manual Override & Fallback)

- **관련 변수**: `clockOut == null` , `isEditedByBoss`
- **상황 (Scenario)**: 알바생이 너무 바쁘거나 실수로 출근 바코드를 안 찍었거나, 퇴근할 때 앱을 켜서 퇴근 버튼을 누르는 것을 잊어버린 경우.
- **구현 내용**:

- **퇴근 미점오 (Null ClockOut) 방어**: 퇴근 기록이 없는 상태(`clockOut == null`)로 급여 정산일이 도래하더라도, 알바페이 계산 엔진이 다운되거나 전체 급여가 0원으로 처리되지 않도록 안전장치(Fallback)가 구현되어 있습니다. 시스템은 미완성된 기록을 비정상(결근 또는 정산 보류)으로 분류하여 급여 과다 지급을 막습니다.
- **사장님 수동 보정 기능 (`isEditedByBoss`)**: 알바생이 기록을 누락했더라도 걱정할 필요가 없습니다. 사장님 앱의 '출퇴근 관리' 메뉴에서 사장님이 직접 해당 알바생의 실제 출퇴근 시간을 수동으로 입력하거나 수정할 수 있는 권한이 있습니다.
- 사장님이 수동으로 시간을 입력하면 `isEditedByBoss = true` 라는 꼬리표가 붙어, 해당 기록이 알바생이 직접 찍은 원본인지 사장님이 사후에 보정한 것인지 노무적 증빙 자료로 명확히 남게 됩니다. 이를 통해 추후 발생할 수 있는 근로 시간 분쟁을 예방합니다.

2.3. 동적 법정 최소 휴게시간 강제 보장 로직 (Dynamic Legal Floor)

- **관련 변수**: `breakMins` , `actualStay` (체류시간), `legalMinimum` , `calculateAppliedBreak`
- **구현 내용**:
  - 알바생이 대타 근무를 하거나 무단으로 연장 근무를 했을 때, 원래 스케줄과 휴게시간이 겹치지 않으면 시스템이 0분을 차감할 수 있는 위험을 방지하였습니다.
  - **[법정 최소치 강제 덮어쓰기]**: 시스템이 계산한 휴게시간이 노동법 최저 기준에 미달할 경우 강제로 상향 조정합니다.
    - 실제 근무시간(`actualStay`) >= 8시간 ➔ 무조건 최소 60분 차감
    - 실제 근무시간(`actualStay`) >= 4시간 ➔ 무조건 최소 30분 차감
  - 이 로직을 통해 근로기준법상 의무 휴게시간 부여가 누락되는 리스크를 줄입니다.
  - **[유급 휴게시간(isPaidBreak = true) 설정 시 방어 및 보존 로직]**:
    - **근로시간 분리 (노무적 방어)**: 유급 휴게로 설정하더라도, 8시간 체류 시 엔진은 "순수 근로시간 7시간 + 휴식 1시간"으로 강제 분리 차감하여 기록합니다. 이는 근로기준법 제54조(의무 휴게시간 부여) 위반(불법) 리스크를 완전히 차단하기 위함입니다.
    - **유급 수당 환원 (급여 보존)**: 강제로 차감되었던 1시간의 휴게시간은 버려지지 않고, 정산 시점에 시급을 곱하여 별도의 ** `유급 휴게수당(breakPay)` **으로 자동 생성됩니다. 최종 급여에는 [기본급(7h) + 유급 휴게수당(1h)] 형태로 합산되어, 사장님과 직원이 약속한 대로 알바생의 총액 급여는 1원도 깎이지 않고 100% 보존됩니다.

2.4. 스케줄이 없는 날에 근무를 하는 경우 (Unscheduled Shifts / 대타 근무)

- **상황 (Scenario)**: 알바생이 원래 쉬는 날이거나, 근무 스케줄표에 아예 등록되어 있지 않은 날에 매장에 나와 출근 바코드를 찍고 일한 경우 (예: 대타, 행사 지원 등).
- **구현 내용**:
  - **보정 로직 미적용 (실시간 인정)**: 스케줄이 없기 때문에 "정해진 출근 시간보다 얼마나 일찍 왔는지", "퇴근 시간보다 늦게 갔는지"를 판별할 기준점(`scheduledShiftStartIso` , `scheduledShiftEndIso = null`)이 존재하지 않습니다. 따라서 2.1항의 **'조기 출근 캡(Cap)'이나 '지연 퇴근 절삭' 방어막이 작동하지 않으며, 알바생이 바코드를 찍은 실제 출퇴근 시간이 그대로 근로시간으로 산출**됩니다.
  - **허위/무단 출근 통제 권한**: 근로자가 사장님의 지시 없이 스케줄 없는 날에 출근을 찍은 경우, 해당 기록은 `attendanceStatus = 'Unplanned'` 으로 저장되어 **사장님이 승인하기 전까지 급여 정산에서 자동 제외**됩니다. 사장님이 '출퇴근 관리' 메뉴에서 해당 기록을 삭제하거나, 승인 대기 상태를 유지하여 급여에 포함되지 않도록 통제할 수 있습니다.

---

3. 기본 급여 산정 로직 (시급제 / 월급제 분리 적용)

3.1. 시급제 기본 급여 산정

- **관련 변수**: `pureMinutes` , `hourlyRate` , `basePay` , `isPaidBreak` , `breakPay`
- **구현 내용**:
  - **정확한 분 단위 산정**: 알바생의 실 근로시간(총 체류시간 - 법정/설정 휴게시간)을 분(`pureMinutes`) 단위로 계산한 뒤, 설정된 시급(`hourlyRate`)을 곱하여 기본급을 산출합니다.
  - **최저임금 미달 입력 제한 (Hard Block)**: 근로자를 등록하거나 시급을 수정할 때, 입력한 시급이 당해 연도 최저임금(예: 2026년 10,320원) 미만일 경우 시스템이 경고창을 띄우며 저장을 제한합니다. 이를 통해 사장님이 실수로 법에 어긋나는 시급을 설정하는 리스크를 줄이도록 지원합니다.
  - **최저임금 강제 소급 적용 (체불 방어)**: 해가 바뀌어 새로운 최저임금이 적용되는 시점부터는, 사장님이 실수로 알바생의 시급 인상(업데이트)을 잊었더라도 계산 엔진이 출근 연도를 스스로 스캔하여 **근로기준법상 새로운 최저임금액으로 강제 상향(소급) 적용**하여 계산합니다. 이를 통해 사후에 발생할 수 있는 임금 체불 리스크를 줄이는 데 기여합니다.
  - **유급 휴게시간 지원**: 만약 근로계약 상 휴게시간을 유급으로 처리하기로 한 경우(`isPaidBreak = true`), 차감되었던 휴게시간만큼 별도의 유급 휴게수당(`breakPay`)으로 산출하여 기본급에 합산해 줍니다.

3.2. 월급제 기본 급여 산정 및 포괄임금제 특례 (Monthly Pro-rata & Inclusive Wage)

월급제 직원은 매월 근무일수나 주말의 일수와 무관하게 고정된 급여를 받는 것을 원칙으로 하나, 결근, 중도 입퇴사, 초과 근무 발생 시 노동법에 어긋나지 않도록 정밀한 보정 엔진이 가동됩니다.

- **관련 변수**: `baseSalary`, `proRataRatio`, `mealPay`, `fixedOvertimePay`, `minimumWageHardBlock`
- **구현 내용**:
  - **포괄임금 마법사 (Auto-Wizard Target Salary) 및 다중 수당 분배**: 사장님이 직원과 '월 250만 원' 등 총액(Target Salary)으로 계약을 체결할 경우, 시스템이 알바생의 주당 소정근로시간을 분석하여 해당 총액을 1차적으로 **'기본급'과 '비과세 식대(최대 20만 원)'** 로 자동 분할(역산)합니다. 만약 법정 수당을 제외하고도 목표 총액에 도달하기 위해 잔여 금액이 남는다면, 이를 **'고정연장수당', '직책 수당', '성실 수당', '기타 커스텀 수당'** 등으로 자유롭게 배분하여 등록할 수 있도록 다중 수당 분배(Multi-Allowance Distribution) UI를 제공합니다. 이를 통해 세금을 절세하면서도 사업장별로 유연한 포괄임금제 계약서 작성이 원클릭으로 완료됩니다.
  - **고정연장수당 초과분 자동 보전 (Overtime Protection)**: 포괄임금제 계약이더라도 알바생이 실제로 일한 시간의 법적 가치가 고정수당액을 넘어서면 임금 체불이 됩니다. 알바페이 엔진은 매월 정산 시 아래와 같은 정밀 추적 로직을 가동하여 **초과분(차액)을 급여에 자동 합산**합니다.
    - **초과 시간 산출**: 월간 실제 발생한 순수 연장근로시간(`overtimeHoursOnly`)에서 계약서상 약정된 고정 연장시간(`fixedOTHours`)을 차감하여 초과 연장근로시간(`fixedOvertimeExcessHours`)을 도출합니다.
    - **[상시근로자 5인 이상 사업장]**: 초과 수당 가산율이 1.5배로 적용되며 야간 가산(0.5배)이 독립적으로 합산됩니다.
      - **산식**: `[초과 연장시간 × 통상시급 × 1.5] + [야간근로시간 × 통상시급 × 0.5]`
    - **[상시근로자 5인 미만 사업장]**: 근로기준법상 가산수당 의무가 없으므로 연장 가산율은 1.0배(기본급 상당액)로 적용되며, 야간 가산은 0원으로 처리됩니다.
      - **산식**: `[초과 연장시간 × 통상시급 × 1.0] + [야간근로 가산 없음 (0원)]`
    - **야간 가산 독립 산정 (5인 이상)**: 포괄임금의 고정수당은 보통 '연장근로'만을 커버하므로, 알바생이 밤 10시 이후 근무하여 발생하는 '야간수당 가산분(0.5배)'은 고정시간 초과 여부와 무관하게 전액 별도로 산출되어 초과 보전금(`fixedOvertimeExcessPay`)에 합산 지급됩니다.
  - **통상임금(Ordinary Wage) 정밀 산정 기준**: 연장/야간/휴일 등 초과 수당을 계산하는 기준값이 되는 '통상시급'을 엔진이 법적 기준에 맞춰 분리 산출합니다.
    - **식대의 통상임금 포함**: 매월 정기적·일률적으로 지급되는 비과세 식대(`mealPay`)는 세법상 비과세일 뿐, 노동법상으로는 통상임금에 무조건 포함되어야 합니다. 알바페이 엔진은 수당 계산 시 `(기본급 + 식대) ÷ 월 소정근로시간` 공식을 적용하여 통상시급(`conservativeHourly`)을 산출함으로써, 사장님이 식대를 제외하고 수당을 적게 계산하여 발생할 수 있는 체불 리스크를 줄여줍니다.
    - **[대법원 전원합의체 판례 적용] 정기성·일률성·고정성**: 2013년 대법원 전원합의체 판결(2012다89399)에 따라 명칭 여하를 불문하고 정기적, 일률적, 고정적으로 지급되는 임금은 통상임금에 해당합니다. 알바페이는 이를 코드로 구현하여 식대(`mealPay`), 직책수당 등 기본급 외의 고정 수당들을 통상임금 베이스(`conservativeHourly`)에 모두 합산시킴으로써, 가산수당 산정 시 기준 시급이 축소되어 체불이 발생할 수 있는 문제를 방지하고자 합니다.
    - **고정연장수당의 합법적 제외 조건 명시 (통상임금 산입 분쟁 방어)**: 매월 고정적으로 지급되는 연장수당은 자칫 '정기성·일률성·고정성'을 갖춘 것으로 오인되어 통상임금에 산입해야 한다는 노무적 분쟁(추가 가산수당 청구 소송)의 타겟이 되기 쉽습니다. 그러나 대법원 판례상 **근로계약서에 '기본급'과 '연장근로에 대한 대가(시간 명시)'가 명확히 구분되어 기재된 포괄임금 계약**의 경우, 해당 수당은 소정근로의 대가가 아니므로 통상임금에서 적법하게 배제됩니다. 알바페이는 '포괄임금 마법사'를 통해 해당 연장 시간을 명확히 역산해 내고, 이를 **앱 내 '스마트 근로계약서'에 별도 항목과 시간으로 분리 명시(Documentation)**함으로써 이러한 법적 논란의 발생 가능성을 낮추고 사장님의 과다 지출 리스크를 줄여줍니다.

> ⚠️ ***통상임금 산정 참고 사항*** *본 엔진의 통상시급(`conservativeHourly`)은* ***기본급과 식대****를 기준으로 산출됩니다. 정기상여금·성과급·직무수당·직책수당 등 사업장별 고유 수당은 반영 범위에 포함되지 않으며, 이러한 항목의 포함 여부에 따라 실제 통상임금은 달라질 수 있습니다. 보다 정확한 통상임금 판단을 위해* ***노무사 또는 전문가의 자문을 권장****합니다.*

> 📌 ***포괄임금 오남용 방지 지침 관련 참고 (고용노동부 '26.4.9~)***
>
> *고용노동부 지침 핵심 요구사항: ① 실근로시간 측정 의무 ② 약정 초과분 추가 지급 의무*
>
> ***알바페이가 제공하는 보조 기능:***
>
> - *바코드 출퇴근 시스템을 통한 실근로시간 기록 지원*
> - *고정OT 약정시간 초과 시 차액 산출 보조*
> - *급여 명세서에 기본급 / 연장수당 / 야간수당 / 주휴수당 항목별 분리 표시*
>
> *위 기능은 지침 준수를* ***보조하기 위한 도구****이며, 최종적인 법적 판단과 책임은 사업주에게 있습니다. 구체적인 적용 방법은 노무사 자문을 권장합니다.*

> 💡 ***[실사례 적용 가이드] 가상데이터 '김점장'의 포괄임금제(월급제) 정산 시뮬레이션***

> ***[계약 조건]***
>
> - ***총 월급(Target Salary)**: 2,500,000원 (상시 5인 이상 매장)*
> - ***스케줄**: 법정 근로 주 40시간 + **월 8.4시간 고정 연장** (초과 시 수당 지급)*
>
> ***1. 포괄임금 마법사 자동 분할 (엔진 내부 처리)***
>
> - *알바페이 마법사는 250만 원을 역산하되, 최저임금 미달을 방지하기 위해 **기본급을 무조건 당해 연도 최저시급(2026년 10,320원) 기준인 2,156,880원으로 하드 락(Lock)** 합니다.*
> - *남은 금액 중 **200,000원은 비과세 식대**로 자동 배정하여 사장님과 직원의 세금 부담을 최소화합니다.*
> - *나머지 **143,120원은 고정연장수당**으로 배정됩니다.*
> - *검증: (기본급 2,156,880 + 식대 200,000) ÷ 209 = **통상시급 11,276원**. 통상시급 기준 1.5배(16,914원)로 고정연장수당 14.3만 원을 역산하면 **월 8.46시간**의 연장근로를 합법적으로 커버합니다.*
>
> ***2. 정상 근무 달의 급여 (오차율 0%)***
>
> - *김점장이 월 8.4시간 이내의 연장근로만 수행하며 스케줄대로 출퇴근했다면, 엔진은 초과 연장 없이 **정확히 2,500,000원**을 산출합니다.*
>
> ***3. 바쁜 달의 초과 연장근로 보전 (Overtime Protection)***
>
> - *이번 달에 손님이 많아 김점장의 실제 총 연장근로가 **30시간**으로 찍혔다고 가정해 보겠습니다.*
> - *고정수당이 이미 커버하는 시간(8.4시간)을 초과한 **21.6시간**에 대해 엔진이 즉각 개입합니다.*
> - *추가 지급액 = `21.6시간 × 11,276원(통상시급) × 1.5배(가산율)` = **약 365,342원**을 250만 원에 자동으로 얹어서 명세서를 발행하여 임금 체불 리스크를 줄입니다.*
>
> ***4. 스케줄 없는 휴무일(대타) 출근 자동 분류***
>
> - *만약 김점장이 원래 쉬는 날(근무시간표상 스케줄이 없는 날)에 매장이 바빠 대타로 출근했다면, 엔진은 해당일의 근태 기록을 스케줄표와 대조하여 즉시 **'휴무일 근로'**로 자동 분류합니다.*
> - *휴무일에 일한 시간은 소정근로시간이나 고정연장수당 대상에 포함되지 않고, **별도의 휴일근로수당(`offDayWorkPay`)으로 분리 정산**됩니다. 엔진은 `offDayHours × 통상시급 × 휴일가산율(5인 이상: 1.5배, 미만: 1.0배)` 을 월급 총액에 별도로 얹어 지급합니다. 이를 통해 고정연장수당은 평일 잔업에만 정확히 적용되고, 휴무일 출근분은 독립적으로 보전되어 무상 노동 발생 리스크를 줄입니다.*
>
> ***5. 중도 입사 월의 일할 계산 (Pro-rata)***
>
> - *김점장이 해당 월의 16일에 입사하여, 그 달 총 30일 중 15일만 재직(출근)했습니다.*
> - *엔진은 재직 비율(`15/30 = 0.5`)을 산출하여, **기본급 50%, 식대 50%, 고정연장수당 50%**를 각각 비례 산정하여 **1,250,000원**으로 일할 계산을 완료합니다.*

- **최저임금 미달 방지 이중 락(Lock)**:
  - **1단계 (Hard Block)**: 월급제 직원을 등록할 때, 분할 산출된 기본급을 정부 기준 월 소정근로시간(예: 209시간)으로 나누어 검증합니다. 통상시급이 당해 연도 법정 최저시급(예: 2026년 10,320원)에 미달하면 앱에 저장을 제한합니다.
  - **2단계 (Warning)**: 근로시간 변동 등으로 인해 기본급과 식대를 합친 금액이 최저임금 기준액(2,156,880원)에 미달할 경우 급여 대시보드에 강력한 경고 알림을 띄웁니다.
- **정교한 일할 계산 (Pro-rata Ratio) 및 결근 공제**:
  - **중도 입/퇴사**: 월 중도에 입사하거나 퇴사하여 한 달 만근을 채우지 못한 경우, 해당 월의 전체 일수 대비 **실제 재직한 일수의 비율(`proRataRatio`)을 소수점 아래까지 정확히 산출**하여 기본급, 식대, 고정수당을 모두 일할 계산하여 깎아냅니다.
  - **결근 감지 및 주휴수당 자동 차단**: 월급제 직원에게 스케줄이 있는 날 출근 기록이 없으면, 엔진은 해당 주의 주휴수당을 즉시 차단(`weeklyHolidayBlockedByAbsence`)합니다. 단, 사장님이 급여 대시보드에서 **[지급 승인]** 스위치를 켜면 차단이 해제되어 주휴수당이 정상 지급됩니다(§5.2 참조). 또한 **기본급 자체는 자동 공제하지 않습니다.** 이는 시스템이 무단결근/사장님 지시 휴업(휴업수당 70% 지급 의무, 제46조)/바코드 미점오/유급 경조사 등을 구별할 수 없기 때문입니다. 사장님 앱 급여 대시보드에 **적색 경고 카드(⚠️ 결근 감지)**를 표시하여, 결근 사유를 사장님이 직접 확인 후 필요 시 `전월 정산금(previousMonthAdjustment)` 기능으로 수동 공제할 수 있도록 안내합니다.
- **근로자의 날(5/1) 및 법정 공휴일 유급휴일 자동화**:
  - **월급제 근로자**: 이미 한 달 치 월급 안에 근로자의 날 유급휴일 수당이 포함되어 있으므로 쉴 경우 별도 수당이 이중 지급되지 않도록 처리합니다. 출근 시에는 휴일근로가산수당만 추가 정산합니다.
  - **시급제(시간제) 근로자**: 5월 1일에 **출근하지 않고 쉬더라도 하루 치 유급휴일수당(통상 1일 소정근로시간분)을 자동 지급**하여 체불 리스크를 줄입니다. (단, 입사일이 5/1 이후인 직원은 자동 제외)
  - **상시 근로자 수(5인 이상/미만)에 따른 가산율 자동 차등 적용**: 만약 5월 1일에 **실제로 출근했다면**, 엔진은 상시 근로자 수 설정을 확인하여 자동으로 다음과 같이 정산합니다.

    - `5인 이상 사업장` : 쉰 것으로 치고 받는 유급휴일수당(100%)에 더해, 실제 일한 시간에 대한 **휴일근로 가산수당(1.5배)**을 얹어주어 총 250%의 임금을 합산합니다.
    - `5인 미만 사업장` : 가산수당 지급 의무가 없으므로 유급휴일수당(100%)에 더해, 실제 일한 시간에 대한 **기본시급(1.0배)**만 정직하게 추가 계산하여 총 200%의 임금을 산출합니다.
  - **[향후 업데이트 예정]**: 현재 근로자의 날(5/1) 로직이 구현되어 있으며, 향후 **공공데이터포털(특일정보 API)과 연동하여 달력상의 모든 빨간 날(법정 공휴일)에 대한 휴일근로수당 자동 적용 및 대체공휴일 로직이 엔진에 추가로 탑재될 예정**입니다.

3.3. 수습기간(Probation) 감액 적용 및 최저임금 연동

- **관련 변수**: `isProbation`, `probationMonths`, `probationWage`
- **구현 내용**:
  - **수습 90% 자동 계산 지원**: 1년 이상의 근로계약을 체결하는 경우, 사장님이 원한다면 최초 1~3개월간 수습기간을 설정하여 원래 시급/월급의 90%만 지급하도록 설정(`isProbation = true`)할 수 있습니다. 출퇴근 일자 및 급여 정산 기간을 기준으로 90% 감액된 금액으로 정산되며, **월급제 직원의 수습기간이 월 중간에 종료되는 경우 '수습 일수'와 '정상 일수'를 분리하여 정교한 일할(비례) 계산**이 자동 적용됩니다.
- **법적 방어막 (Legal Defense Mechanism)**:
  - **최저임금 미달 경고 (Compliance Warning)**: 수습 90% 감액을 적용한 시급(`probationWage`)이 당해 연도 법정 최저임금보다 낮아질 경우, 시스템이 즉각 **"⚠️ 최저임금법 위반입니다"**라는 강력한 경고를 띄웁니다. 이를 통해 수습기간이라 할지라도 '단순노무직(예: 편의점, 패스트푸드 등)'이 아니며 '1년 이상 계약'이어야만 90% 감액이 합법이라는 노무적 리스크를 사장님께 환기시킵니다.
  - **1년 미만 계약자 수습 차단 (Contract Duration Lock)**: 근로기준법상 1년 미만의 단기 근로계약을 체결한 근로자에게는 수습기간을 이유로 한 최저임금 감액(90%) 적용이 법적으로 전면 금지되어 있습니다. 이를 반영하여 앱 내 직원 등록 화면에서 **계약 기간(입사일 ~ 종료일)이 1년(365일) 미만으로 설정된 경우, '수습기간 적용(90%)' 스위치 자체가 눌리지 않도록 제한**했습니다. 만약 1년 이상으로 설정하여 수습을 켜둔 상태에서 캘린더를 통해 계약 기간을 1년 미만으로 단축 변경할 경우에도 내부적으로 수습 상태(`isProbation`)가 강제로 해제(`false`)되어 사장님의 의도치 않은 법 위반 가능성을 시스템 레벨에서 줄여줍니다.
  - **최대 3개월 System Lock**: 법적 수습기간 인정 한도인 최대 3개월을 초과하여 설정할 수 없도록, 앱 UI 레벨에서 수습기간을 1~3개월까지만 선택 가능하게 제한(Lock)해 두었습니다.

---

4. 연장, 야간, 휴일근로 가산수당

상시근로자수(`isFiveOrMore`)에 따라 엔진의 가산율 분기가 설계되어 있습니다.

4.1. 수당별 변수 및 산식

- **연장근로수당 (`totalOvertimeMinutes`)**:
  - 하루 8시간 초과 근무 시 발생. 5인 이상은 1.5배, 5인 미만은 1.0배(기본급만) 지급.
- **야간근로수당 (`totalNightMinutes`)**:
  - 밤 10시 ~ 다음날 아침 6시 사이 근무 시 발생. 5인 이상은 0.5배 야간 가산액 지급, 5인 미만은 가산 의무가 없어 0원으로 처리.
- **포괄임금제 초과 연장분 보전 (`fixedOvertimeExcessPay`)**:
  - 월급제(포괄임금) 직원이 계약서에 약정된 고정 연장시간을 넘겨서 일한 경우, 넘친 시간만큼 추적하여 차액을 자동 보전해 줍니다.

---

5. 주휴수당 및 근로자의 날(5/1) 유급휴일 특례

5.1. 주휴수당 산출 로직

- **관련 변수**: `isWeeklyHolidayEligible`, `weeklyHolidayBlockedByAbsence`
- **구현 내용**:
  - **지각/조퇴 보호**: 직원이 지각이나 조퇴를 해서 주 15시간을 채우지 못했더라도, 노동법상 결근이 아니므로 주휴수당을 뺏지 않고 정상 지급합니다. (지각한 시간만큼의 시급만 차감)
  - **무단 결근 차단 및 리스크 방어 UI 알림**: 소정근로일에 출근을 안 한 결근 감지 시, `weeklyHolidayBlockedByAbsence` 변수가 작동하여 그 주의 주휴수당 지급을 즉시 차단(Block)합니다. 동시에 급여 명세서 화면 최상단에 **강력한 적색 경고 카드(⚠️ 결근 감지)**를 띄워 사장님께 차감 사실을 명확히 고지합니다. 사장님이 **[지급 승인]** 스위치를 켜면 차단이 해제되어 주휴수당이 정상 지급됩니다(§5.2 참조).
    - **시급제**: 주휴수당이 별도 항목(`weeklyHolidayPay`)으로 합산되므로, 스위치 OFF 시 해당 주 주휴수당이 **자동으로 0원 처리**(실질 공제)됩니다.
    - **월급제**: 주휴수당이 이미 기본급(`baseSalary`)에 내포되어 있으므로, 스위치 OFF/ON과 무관하게 **월급 자체는 변동되지 않습니다.** 결근 기본급 공제가 필요한 경우 사장님이 `전월 정산금(previousMonthAdjustment)` 기능으로 수동 차감합니

다.
    - 경고 카드에는 *"매장 사정으로 쉬게 한 휴업일 경우에는 주휴수당/휴업수당을 수동으로 지급해야 한다"*는 가이드라인을 함께 명시하여, 사장님이 의도치 않게 임금 체불(법 위반)이 발생할 리스크를 줄여줍니다.

5.2. 초단시간(15시간 미만) 근로자 주휴수당 특례 및 수동 부여 (isApprovedException)

- **관련 변수**: `manualWeeklyHolidayApproval`, `isApprovedException`
- **구현 내용**:
  - 원칙적으로 주 15시간 미만의 소정근로시간으로 계약된 초단시간근로자는 근로기준법상 주휴수당 및 연차유급휴가 적용 대상이 아닙니다(제18조 제3항). 따라서 엔진은 단발성 대타·연장근로 등으로 실제 근로시간이 일시적으로 증가하더라도, **근로계약상의 소정근로시간을 기준으로 주휴 및 연차 발생 여부를 우선 판단**합니다. 다만 실제 근로가 장기간 반복적으로 15시간 이상 지속되는 경우에는 실질 근로형태에 대한 별도 노무 검토가 필요할 수 있습니다.
  - 하지만 사장님이 선의로 지급을 원하시거나 합의에 의해 소정근로시간이 일시 변동되었다고 판단하실 경우, 앱 내 급여 대시보드에서 **[지급 승인]** 스위치를 켤 수 있습니다.
  - 이 스위치가 켜지면 시스템은 원래 계약시간(예: 11시간)의 삭감 한도를 즉각 해제하고, **그 주에 실제 일한 시간(최대 40시간 한도)에 비례하여 정당한 주휴수당을 온전하게 계산**하여 총액에 합산합니다.

5.3. 근로자의 날(5월 1일) 특례 로직 (알바생 비례 산정)

- **관련 변수**: `laborDayAllowancePay`, `weeklyHoursPure`
- **구현 내용**:
  - **미근무 시 유급휴일 보장**: 5월 1일에 일하지 않아도, 단시간 근로자(알바생)의 주간 소정근로시간에 비례하여 **`(주간 소정근로시간 ÷ 40시간) × 8시간 × 시급`**의 공식으로 법정 보장액을 자동 계산해 급여에 얹어줍니다.
  - **근무 시 휴일 가산**: 만약 5/1에 실제 출근까지 했다면, 위 보장액에 더해서 [일한 시간 × 시급 × 휴일가산율(1.5배 또는 1.0배)]이 별도로 추가 지급됩니다.

---

6. 연차 휴가 및 퇴직금 (더블 디스카운트 원천 방어)

6.1. 연차 저금통 및 소멸·사용촉진 만료 처리

- **관련 변수**: `annualLeaveAllowancePay`, `hoursMultiplier`
- **구현 내용**:
  - **연차 발생 규칙 (근로기준법 제60조)**:
    - **1년 미만** (제60조 제2항): 1개월 개근 시 1일의 유급휴가 발생 (최대 11개).
    - **1년 이상** (제60조 제1항): 1년간 출근율 80% 이상 시 15일 발생. 80% 미만 시 해당 연도 15일 **차단** (1년 미만분은 유지).
    - **근속 가산** (제60조 제4항): 3년 이상 계속 근로 시, 최초 1년을 초과하는 매 2년마다 1일 가산 (상한 25일).
    - **적용 제외**: 5인 미만 사업장(법적 의무 없음), 초단시간 근로자(주 15시간 미만, 제18조 제3항).

근속 연차	발생일수	가산일	근속 연차	발생일수	가산일
1년차	15일	+0	14년차	21일	+6
2년차	15일	+0	15년차	22일	+7
3년차	16일	+1	16년차	22일	+7
4년차	16일	+1	17년차	23일	+8
5년차	17일	+2	18년차	23일	+8
6년차	17일	+2	19년차	24일	+9
7년차	18일	+3	20년차	24일	+9
8년차	18일	+3	21년차+	**25일 (상한)**	+10

- **단시간 연차 가치 보정 (분쟁 예방 지원)**: 단시간 근로자에게 연차 부여 시 일수를 깎고 하루치 시간도 깎아버리는 '이중 차감(Double Discount)' 리스크를 줄입니다. 발생이 확정된 연차 일수(출근율 충족 시 15일)를 정상 부여하되, 사용할 때 하루 치 가치 (`hoursMultiplier`)만 비례로 환산합니다.

6.1.1. 연차 사용촉진제도 구현 (근로기준법 제61조)

법적 근거:

- **제61조 제1항 (정기 연차)**: 연차 소멸 **6개월 전**까지 1차 촉진(사용 시기 지정 요청), 소멸 **2개월 전**까지 2차 촉진(사용 시기 직접 지정)을 서면으로 이행하면, 소멸된 미사용 연차에 대한 수당 지급 의무가 면제됩니다.
- **제61조 제2항 (1년 미만 연차)**: 소멸 **3개월 전**까지 1차 촉진, 소멸 **1개월 전**까지 2차 촉진 완료 시 동일한 면제 효과가 적용됩니다.
- **핵심**: 촉진 절차를 적법하게 이행하면 사용자(사장님)는 해당 소멸 연차에 대한 수당 지급 의무에서 면제될 수 있습니다. 다만, 서면 통보의 적법성·시기 등에 대한 최종 판단은 사안별로 달라질 수 있으므로, 중대한 사안의 경우 전문가 자문을 권장합니다.

엔진 구현 상세 (`LeavePromotionStatus` 모델):

필드	역할
`batchGrantDate` / `batchExpiryDate`	연차 배치 발생일·소멸일 (촉진 대상 식별)
`isPreAnniversary`	1년 미만 발생분 여부 → 촉진 기한 유형 자동 분기 (3개월/1개월 vs 6개월/2개월)
`firstNoticeDeadline` / `secondNoticeDeadline`	엔진이 자동 계산하는 1차·2차 촉진 기한
`firstNoticeDate` / `firstNoticeDocId`	1차 촉진 통보 완료일 및 PDF 문서 ID
`secondNoticeDate` / `secondNoticeDocId`	2차 촉진 통보 완료일 및 PDF 문서 ID
`employeePlanDate` / `employeePlanContent`	직원의 사용 계획 제출 내역
`designatedDates`	2차 촉진 시 사장님이 지정한 사용 날짜 목록
`status`	상태 흐름: `pending` → `first_sent` → `awaiting_plan` → `second_sent` → `completed` / `expired`
`auditHash` / `auditTimestamp`	SHA-256 무결성 해시 (위변조 방지)

촉진 자동 감시 로직:

1. **배치별 촉진 윈도우 진입 감지**: 엔진(`AnnualLeaveCalculator`)이 급여 정산 시 각 연차 배치의 `firstNoticeDeadline` 을 현재 날짜와 비교하여, 촉진 윈도우에 진입한 배치를 자동으로 `promotionStatuses` 목록에 추가합니다.
2. **기한 경과 자동 상태 전환**: 소멸일이 경과했는데 촉진이 미완료인 배치는 자동으로 `expired` 상태로 전환되며, 해당 수당이 지급 필요 금액으로 자동 산정됩니다.
3. **촉진 완료 시 수당 면제 반영**: `status == 'completed'` 인 배치의 소멸분은 `promotionExemptPayoutAmount` 에 합산되어, 급여 명세서에서 "💰 촉진 완료 수당 면제" 항목으로 자동 표시됩니다.

급여 명세서 UI 표시:

- 연차 저금통 섹션 하단에 **[연차 사용촉진 현황]** 카드가 자동 표시됩니다.
- 배치별로 상태 배지(⏳ 촉진 미이행, 📩 1차 통보 완료, ✅ 촉진 완료, ❌ 기한 만료)가 표시되며, 1차/2차 촉진 기한이 명시됩니다.
- 사장님은 이 현황을 통해 어떤 배치에 대해 어떤 단계의 촉진을 언제까지 이행해야 하는지 한눈에 파악할 수 있습니다.

6.1.2. 연차 사용 등록 및 급여 엔진 반영 흐름 (근로기준법 제60조 제6항)

법적 근거:

- **제60조 제6항**: "연차유급휴가를 사용한 기간은 출근한 것으로 본다" → 연차 사용일은 결근이 아닌 **출근으로 간주**되어, 주휴수당 만근 판정에 불이익이 없어야 합니다.

4단계 데이터 흐름:

단계	동작	저장소	관련 모듈
①	`ScheduleOverride` 저장 (`leaveType: 'annual_leave'`)	Hive (로컬)	`_handleAnnualLeave()`
②	`Attendance` 레코드 생성 (`isAttendanceEquivalent: true`, `clockIn == clockOut`)	Firestore	`DatabaseService.recordAttendance()`
③	`worker.usedAnnualLeave += 1.0`	Hive → Firestore	`WorkerService.save()`
④	Firestore `rosterDay` 동기화 (`isOff: true`)	Firestore	`_syncRosterDayToFirestore()`

핵심 방어 메커니즘:

1. **이중 지급 방지 및 연차 유급 보장**: 연차 Attendance는 `clockIn == clockOut` 형태의 특수 레코드로 생성되며, 실제 근로시간 계산 루프에서는 `stayMinutes = 0 → continue` 처리되어 **근로시급 계산 대상에서는 제외**됩니다. 다만 연차 사용일은 근로기준법상 유급휴가이므로, 급여 엔진은 별도의 `annualLeaveUsedPay` 항목을 통해 **'1일 소정근로시간 × 통상시급'** 기준의 연차유급수당을 지급합니다. 이를 통해 연차 유급 보장은 유지하면서도, 실근로임금과 연차수당의 이중 지급을 방지합니다.
   - **1일 소정근로시간 산출**: `주당 소정근로시간(weeklyHoursPure) ÷ 주당 근무일수(scheduledWorkDays.length)`
   - **시급 소급 적용**: `_getHourlyWageForDate()` 를 통해 연차 사용일 기준 시급을 적용 (임금 변경 이력 반영)
   - **최저임금 하한 보장**: 연차일 기준 시급이 법정 최저임금 미만이면 자동 상향
   - **수습기간 90% 적용**: 수습 중 연차 사용 시 감액 시급 자동 반영
   - **월급제 중복 방지**: 월급제 근로자는 연차 사용일의 임금이 이미 월급에 내포되어 있으므로, `annualLeaveUsedPay` 를 `finalTotalPay` 에서 제외하여 과지급을 방지합니다. 퇴사 시 미사용 잔여 연차 정산분 (`leaveSummary.annualLeaveAllowancePay`)만 추가 지급됩니다.
2. **주휴수당 만근 보호**: 급여 엔진의 `_isPerfectAttendanceForRange()` 메서드는 `a.clockOut != null || a.isAttendanceEquivalent` 조건으로 출근일을 판별합니다. 연차 Attendance는 `isAttendanceEquivalent = true` 이므로 **출근일로 인정**되어 주휴수당 만근 판정에 불이익이 없습니다.
3. **취소 시 완전 복원**: 연차 취소 시 ① Override 삭제, ② Attendance 삭제, ③ `usedAnnualLeave -= 1.0` (`.clamp(0.0, ∞)` 로 음수 방지)의 3가지가 원자적으로 롤백됩니다.

가상 데이터 검증 결과 (8개 테스트 전부 통과):

#	테스트 케이스	기대값	결과
1	연차 사용일 유급 보장 (4일 출근 + 1일 연차)	basePay = 4×8h×10,320 + annualLeaveUsedPay = 8h×10,320	✅
2	연차일 = 출근 간주 → 주휴수당 발생	weeklyHolidayPay = (32/40)×8×10,320 = 66,048원	✅
3	연차 없이 결근 → 주휴수당 0원 (대조군)	weeklyHolidayPay = 0원	✅
4	잔여일수 정확성 (26일 발생 - 3일 사용)	remaining = 23일	✅
5	퇴사 정산 (사용 3일 반영 후 잔여분 × 8h × 시급)	annualLeaveAllowancePay 정확	✅
6	취소 복원 (3일 → 1일 취소 → used 2일)	remaining = 24일	✅
7	음수 차감 방지 (used 0 → 취소 → 0 유지)	clamp(0, ∞) = 0	✅
8	월급제 중복 방지 (연차일 임금 미중복)	annualLeaveUsedPay가 finalTotalPay에 미포함	✅

- **테스트 파일**: `packages/shared_logic/test/compliance/annual_leave_usage_test.dart`
- **실행 명령**: `flutter test test/compliance/annual_leave_usage_test.dart`

[향후 구현 예정] 반차(Half-day Leave) 및 연차+근무 혼합 처리

> 📌 *현재 시스템은 **전일 연차(1.0일 차감)**만 지원하며, 아래 시나리오는 향후 업데이트에서 구현 예정입니다.*

시나리오	설명	구현 범위
반차 (0.5일)	오전 반차 / 오후 반차 선택 → `usedAnnualLeave += 0.5` 차감, 유급수당은 반일치(4h × 시급) 지급	UI + 모델 + 엔진
연차 + 실제 근무 혼합	오전 반차 + 오후 4시간 실근무 → 반차 수당(4h)과 실근로 임금(4h)이 각각 별도 Attendance로 분리 정산	반차 선행 필요

예상 변경 사항:

- `ScheduleOverride.leaveType` 에 `'half_day_am'`, `'half_day_pm'` 추가
- `_handleAnnualLeave()` 다이얼로그에 "전일 / 오전 반차 / 오후 반차" 선택지 추가
- 급여 엔진 `stayMinutes <= 0` 분기에서 반차 감지 시 `annualLeaveUsedPay += (dailyContractHours / 2) × leaveRate`

6.2. 퇴직금 정산 엔진 (v3.0 — 법적 정합성 강화)

- **관련 모듈**: `SeveranceCalculator.calculateExitSettlement()` (severance_calculator.dart)
- **관련 법령**: 퇴직급여보장법 제4조, 근로기준법 제2조

6.2.1. 퇴직금 자격 판정 (퇴직급여보장법 제4조)

조건	판정
재직일수 ≥ 365일 **AND** 주 소정근로시간 ≥ 15시간	✅ 퇴직금 대상
재직일수 < 365일	❌ 미대상 (1년 미만)
주 소정근로시간 < 15시간 (초단시간)	❌ 미대상 (v3.0 추가)

6.2.2. 평균임금 산정: 역월 3개월 기준 (근로기준법 제2조 제1항 제6호)

> ⚠️ ***흔한 오류**: 평균임금을 `총 임금 ÷ 90일` 로 계산하는 사례가 많으나, 이는 **법적으로 틀립니다**. 근로기준법은 "**사유가 발생한 날 이전 3개월간**"이라 규정하며, 이 기간의 **역일수(曆日數)는 월 길이에 따라 변동**합니다.*

퇴사일	역월 3개월 기간	역일수	비고
5월 31일	3/1 ~ 5/31	**92일**	3월(31)+4월(30)+5월(31)
4월 30일	2/1 ~ 4/30	**89일**	2월(28)+3월(31)+4월(30), 비윤년
3월 31일	1/1 ~ 3/31	**90일**	1월(31)+2월(28)+3월(31), 비윤년
9월 30일	7/1 ~ 9/30	**92일**	7월(31)+8월(31)+9월(30)

- **v2.0 (이전 — 폐기됨)**: `exitDate - 90일` (90일 고정 → **위법 소지**)
- **v3.0 (현재)**: `_subtractCalendarMonths(exitDate, 3)` (역월 기준, 말일 보정 포함)
- **영향**: 동일한 임금이라도 역일수가 89일이면 1일 평균임금이 높아지고, 92일이면 낮아짐 → **퇴직금에 직접적 영향**

> ***산식**: `1일 평균임금 = 퇴직 전 역월 3개월간 총 임금 ÷ 해당 기간의 역일수(89~92일)`*

6.2.3. 연차수당 산입: 실제 발생일수 기반

- **v2.0 (이전)**: 15일 고정 가정 → 과대/과소 산정 위험
- **v3.0 (현재)**: `_estimateAnnualLeaveDays()` — 근속연수·5인 이상 여부 반영

상황	v2.0 (이전)	v3.0 (현재)
5인 미만 사업장	15일 (위법)	**0일**
입사 6개월 퇴사	15일 (과다)	**5~6일**
3년차 퇴사	15일 (과소)	**16일**

10년차 퇴사	15일 (과소)	**19일**

6.2.4. 평균임금·통상임금 비교 로직 (근로기준법 제2조 제2항)

> ***근로기준법 제2조 제2항***: *"평균임금이 그 근로자의 통상임금보다 저액일 경우에는 그 통상임금액을 평균임금으로 한다."*

- **엔진 동작**: 산출된 1일 평균임금과 1일 통상임금을 비교하여 `Max(평균임금, 통상임금)` 중 높은 쪽을 퇴직금 기준 단가로 사용합니다.
- **통상임금 산정 기준**: 퇴직금 엔진은 **실제 계약일일근로시간(`weeklyHours` ÷ 주간근무일수) × 시급**으로 1일 통상임금을 산출합니다.
  - 전일제(주 40시간 이상): `1일 통상임금 = 8h × 시급`
  - 단시간(주 15~39시간): `1일 통상임금 = (계약 주간시간 ÷ 주간 근무일수) × 시급`

> 📌 ***v3.1 변경 (초단시간 multiplier 버그 수정)***: *이전 비례 공식 `(weeklyHours/40)×8` 은 연차 비례환산(시행령 별표2)에서는 적합하나, **퇴직금 통상임금 산정 시 과소계산 위험**이 있었습니다.*
>
> | *예시 (주 20h / 주 3일 근무)* | *이전 비례 공식* | *수정 후 실제 계약시간* |
> |---|---|---|
> | *hoursMultiplier* | *(20/40)×8 = 4.0h* | *20÷3 = 6.67h* |
> | *1일 통상임금 (시급 10,320)* | ***41,280원*** | ***68,800원*** |
> | ***퇴직금 차이*** | *과소 산정 위험* ❌ | ***법적 정확*** ✅ |

- **비교 로직이 작동하는 경우**: 3개월간 역일수 대비 실근무일이 적어 평균임금이 산술적으로 낮아지는 경우에 비교 결과 통상임금이 선택될 수 있습니다. 이는 근로 유형(전일제/단시간)에 관계없이 동일하게 적용됩니다.

> ⚠️ ***통상임금 산정 한계 고지 (중요)***
>
> *본 엔진의 퇴직금용 통상임금은 `시급 × 소정근로시간` 의 **단순 환산값**이며, 아래와 같은 항목의 포함 여부에 따라 실제 통상임금은 달라질 수 있습니다.*
>
> | *항목* | *엔진 반영 여부* | *법적 판단 기준* |
> |---|---|---|
> | *기본급(시급)* | ✅ *반영* | *소정근로의 대가* |
> | *식대(고정 지급)* | ✅ *반영 (월급제)* | *정기·일률·고정 → 통상임금 (대법원 2013다89399)* |
> | *직무수당·직책수당* | ⚠️ *앱 등록 시 반영* | *고정적 지급 시 산입 대상* |
> | *고정연장수당(OT)* | ❌ *제외* | *소정근로 대가가 아닌 초과근로 대가 (계약서 분리 명시 시)* |
> | *정기상여금* | ❌ *미반영* | *2024 대법원 전원합의체: 정기·일률·고정 시 산입* |
> | *성과급·인센티브* | ❌ *미반영* | *지급 조건 충족 여부에 따라 판단 상이* |
>
> ***본 문서의 예시 계산은 단순 시급 근로자를 기준으로 하며, 실제 통상임금 산정 시 고정수당·정기상여금 등의 포함 여부에 따라 금액이 달라질 수 있습니다. 중대한 사안의 경우 반드시 노무사 또는 전문가의 자문을 받으시기 바랍니다.***

6.2.5. v3.0 테스트 검증 결과 (8개 시나리오)

#	시나리오	기대값	결과
1	초단시간(주 14h) 400일 근무 → 자격 배제	퇴직금 0원	✅
2	주 15시간 정확히 (경계값) → 자격 있음	퇴직금 > 0	✅
3	5월 31일 퇴사 → 역월 92일 (90일 아님)	오류 없이 산출	✅
4	5인 미만 → 연차수당 산입 0원	퇴직금 ≤ 5인 이상	✅
5	입사 6개월 → 퇴직금 0원	미대상, 연차만 정산	✅
6	365일 경계값 → 자격 있음	퇴직금 > 0	✅
7	단시간(주 20h) → 전일제 대비 축소	20h 퇴직금 < 40h	✅

8	지급기한 = 퇴사일 + 14일	5/15 → 5/29	✅

- **테스트 파일**: `packages/shared_logic/test/compliance/severance_pay_test.dart`
- **실행 명령**: `flutter test test/compliance/severance_pay_test.dart`

---

7. 노무서류(근로계약서) 비대면 스마트 워크플로우

전자서명법과 근로기준법 교부 의무를 동시에 충족하는 스마트 근로계약서 렌더링 로직입니다.

7.1. 서류 체계 — 8종 구조화 문서

#	서류명	DocumentType	전용 UI 화면	비고
1	근로계약서 (전일제)	`contract_full`	`ContractPage`	주 40h 이상 자동 분기
2	근로계약서 (단시간)	`contract_part`	`ContractPage`	주 40h 미만 자동 분기
3	야간·휴일 동의서	`night_consent`	`NightConsentScreen`	근로기준법 제70·71조
4	근로자 명부	`worker_record`	`WorkerRecordScreen`	근로기준법 제41조
5	채용 체크리스트	`checklist`	`HiringChecklistScreen`	내부 관리용
6	임금 변경서	`wage_amendment`	`WageAmendmentScreen`	§7.4 참조
7	사용촉진 1차 통보서	`leave_promotion_first`	`LeavePromotionScreen`	§7.5 참조
8	사용촉진 2차 통보서	`leave_promotion_second`	`LeavePromotionScreen`	§7.5 참조

> ***초단시간 근로자 감지:*** *`DocumentCalculator.isUltraShortTime(weeklyHours)` — 주 15시간 미만일 경우 UI에 `초단시간` 경고 뱃지가 표시되고, 계약서 유형이 자동으로 `contract_part` 로 지정됩니다.*

7.2. 진입 경로 — 위자드 vs 개별 생성

7.2.1. 위자드 (신규 직원 일괄) — `DocumentWizardScreen`

4단계 **순차 강제** 워크플로우:

```
① 근로자 명부 → ② 야간·휴일 동의서 → ③ 채용 체크리스트 → ④ 근로계약서
```

- **접근 제어**: 이전 단계가 `draft` 상태이면 다음 스텝 진입 불가
- **누락 서류 자동 생성**: `_ensureAllDocumentsExist()` 가 기존 직원에 대해서도 미작성 서류를 Firestore에 자동 생성
- **계약서 유형 자동 분기**: `weeklyHours >= 40` → `contract_full`, 미만 → `contract_part`
- **완료 상태 자동 감지**: `_findFirstIncompleteStep()` 이 미완성 단계를 찾아 해당 페이지로 자동 이동

7.2.2. 개별 생성 — `CreateDocumentScreen`

8종 그리드에서 원하는 서류 선택 → Firestore 문서 생성 → 해당 전용 화면으로 즉시 이동. 위자드 없이 단일 서류만 작성할 때 사용합니다.

7.3. 근로계약서 자동 렌더링 — 급여 역산 로직

사장님이 입력한 직원 데이터(`Worker`)를 기반으로 근로기준법 표준 양식에 맞는 계약서 텍스트가 자동 생성됩니다.

7.3.1. 통상시급 역산 (S_Ref 방식)

월급제 직원의 경우, 계약서에 기재할 통상시급을 역산합니다:

```
S_Ref (월 소정근로시간) = (주간근로시간 + 1일 주휴환산) × 4.345 (올림)
통상시급 = (기본급 + 식대) ÷ S_Ref
```

변수	산식	예시 (주 40h, 5일)

`weeklyH`	계약 주간시간	40h
`wdpw`	주간 근무일수	5일
`whH` (1일 주휴환산)	weeklyH ÷ wdpw	8h
`sRef`	ceil((40 + 8) × 4.345)	**209시간**
`ordHourly`	(기본급 + 식대) ÷ 209	—

7.3.2. 고정연장수당 자동 산출

```
5인 이상: 고정OT = 고정OT시간 × 통상시급 × 1.5
5인 미만: 고정OT = 고정OT시간 × 통상시급 × 1.0
```

`isFiveOrMore` 값은 Firestore `stores` 컬렉션에서 실시간 조회합니다.

7.3.3. 특약 자동 삽입 (월급제 전용, `DocumentTemplates`)

특약	내용	5인 분기
[특약 1] 주휴수당	"기본급은 유급주휴수당을 포함하여 산정" + 개근 미충족 시 공제 가능 명시	없음
[특약 2] 고정연장 합의	OT시간 역산값 기재 + 초과분 정산 공식 (①이하/②초과/③휴일)	✅ 5인 이상: 1.5배/2.0배 가산, 미만: 1.0배
[특약 3] 평균주수 합의	"1개월 평균 주수 4.345주 기준" + 결근·지각 공제 가능 명시	없음

> ***고정OT 시간 역산 보수적 처리***: `(OT금액 ÷ (통상시급 × 가산배수)) × 10 → floor → ÷ 10` *— 소수점 1자리에서* ***내림****하여 사업주에게 불리하지 않게 처리 (예: 9.76h → 9.7h로 표시)*

7.4a. 메타데이터 기반 부인방지 서명 체계

수집 데이터 (`SecurityMetadataHelper.captureMetadata`)

서명 버튼 터치 즉시, 5가지 보안 메타데이터가 자동 수집됩니다:

항목	수집 방법	용도
`timestamp`	`DateTime.now().toIso8601String()`	서명 시각 입증
`device`	`DeviceInfoPlugin` (브랜드+모델)	서명 기기 특정
`deviceId`	Android: `androidInfo.id` / iOS: `identifierForVendor`	기기 고유 식별
`ipAddress`	`NetworkInfo().getWifiIP()`	서명 네트워크 위치
`gps`	`Geolocator.getCurrentPosition()` (정확도: medium)	서명 물리적 위치

> *"내가 서명한 적 없다", "나중에 서명을 합성했다" 등의 주장에 대한 기술적 대응력을 높입니다.*

문서 무결성 해시 (`generateDocumentHash`)

```
입력: "$type|$staffId|$content|$dataJson|$createdAt"
출력: SHA-256 해시 (64자 hex)
```

이 해시는 **PDF 하단에 인쇄** + **Firestore `documentHash` 필드에 저장**되어, 추후 문서 내용 위변조 여부를 검증하는 데 사용됩니다.

7.4b. 보관·교부·보존 기한

항목	내용
교부	서명 완료 즉시 사장님 앱 + 알바생 앱 양쪽에 동기화 → 근로기준법 교부의무 이행
보관	Firestore + 로컬 Hive 이중 저장
보존 기한	`DocumentCalculator.calculateExpiryDate()` = 작성일 + **3년** (근로기준법 보존 의무)
PDF 발급	`DocumentExportPage` → 기간별·직원별 그룹핑 → `PdfGeneratorService`로 PDF 생성·공유

PDF 생성 메서드 매핑

서류	`PdfGeneratorService` 메서드
전일제 계약서	`generateFullContract()`
단시간 계약서	`generatePartTimeContract()`
임금명세서	`generateWageStatement()`
임금변경서	`generateWageAmendment()`
야간동의서	`generateNightConsent()`
근로자명부	`generateWorkerRecord()`
친권자 동의서	`generateParentalConsent()`

7.4c. 연소자(만 18세 미만) 자동 판별 및 3중 방어 체계

직원 등록 시 생년월일을 입력하면, 엔진이 만 나이를 자동 산출하여 연소자 여부를 판별하고, 3단계 법적 방어 메커니즘을 실행합니다.

1단계: 자동 판별 (`_calcAge` + `_checkMinor`)

```
만 나이 = 현재 연도 − 출생 연도 (생일 미도래 시 −1)
연소자 = 만 나이 < 18
```

생년월일 DatePicker 선택 즉시 `_checkMinor()` 함수가 자동 호출되어 `_isMinor` 플래그를 설정합니다.

2단계: 연소자 안내 UI 자동 표시

`_isMinor == true` 시 빨간색 경고 패널이 즉시 표시됩니다:

안내 항목	내용	근거
🔴 친권자 동의서	부모님 종이 동의서 + 가족관계증명서 필수	제66조
🔴 야간근로 제한	22:00~06:00 원칙 금지 (고용노동부 인가 시 예외)	제70조
🔴 휴일근로 제한	본인 동의 + 고용노동부 인가 필요	제70조
🔴 근로시간	**1일 7시간, 주 35시간 초과 금지**	제69조

3단계: 야간근로 설정 차단

- **요일별 시간 설정 시**: 22:00~06:00에 걸치는 시간 입력 → SnackBar 경고 + 설정 취소
- **저장(Submit) 시**: 일괄 시간 + 요일별 시간 모두 재검증 → 야간 포함 시 **저장 자체 거부**
- **야간동의서 화면**: `NightConsentScreen` 에서도 생년월일 기반 `_isMinor` 자동 감지 → "연소자·임신 중 근로자는 노동부장관 인가 필요" 경고 표시

7.4d. 친권자(법정대리인) 동의서 — A4 인쇄 양식

연소자를 고용할 때 근로기준법 제66조에 따라 반드시 비치해야 하는 친권자 동의서를 앱 내에서 생성하여 **A4 용지로 인쇄**할 수 있습니다.

워크플로우

```
노무서류 관리 → 직원 선택 → 새 서류 작성 → 친권자 동의서 탭
→ 직원 정보 자동 기입 → PDF 생성 → 시스템 인쇄 다이얼로그 표시
```

- **자동 기입 항목**: 직원 성명, 생년월일, 연락처, 시급, 근무요일, 근무시간, 입사일, 사업장 정보
- **수기 기입 항목** (인쇄 후 부모님이 직접 작성): 친권자 성명, 관계, 생년월일, 주소, 서명

PDF 구성

섹션	내용
1. 연소 근로자	성명, 생년월일, 주소, 연락처 (자동 기입)
2. 근무처 정보	사업장명, 대표자, 주소, 연락처 (자동 기입)
3. 근로 조건	시급, 근무요일, 근무시간, 시작일, "1일 7시간/주 35시간 이내" 명시
4. 친권자 정보	성명, 관계, 생년월일, 연락처, 주소 (빈칸 — 수기 기입)
5. 동의문	법적 동의 문구
법적 유의사항	제66조(동의서+증명서 비치), 제67조(대리체결 불가), 제69조(7h/35h), 제70조(야간 금지)
첨부 서류 체크	☐ 가족관계증명서 1부, ☐ 주민등록등본 1부
서명란	친권자(서명 또는 인) + 사업주 확인(서명 또는 인)

> ***설계 의도**: 전자서명이 아닌 **서면 서명** 방식. 근로기준법 제66조의 "동의서 비치" 의무는 원본 서면을 전제하므로, 인쇄하여 부모님이 직접 서명한 뒤 매장에 보관하는 것이 법적으로 안전합니다.*

7.4e. 출퇴근 기록부 조회 및 PDF 발급

Firestore `attendance` 컬렉션의 실시간 출퇴근 데이터를 기간별로 조회하고, A4 PDF로 자동 변환하여 공유/인쇄할 수 있는 기능입니다.

진입 경로

```
노무서류 관리 → 직원 카드 펼침 →「출퇴근 기록부 확인 및 PDF 발급 🖨」버튼
```

직원별 ExpansionTile 하단에 **상시 노출**되는 버튼으로 진입합니다 (`AttendanceRecordScreen`).

데이터 로드 파이프라인

```
Firestore 'attendance' 컬렉션
  → storeId == 현재 매장
  → staffId == 해당 직원
  → clockIn 날짜 ∈ [startDate, endDate]
  → clockIn 기준 오름차순 정렬
```

- **기본 조회 범위**: 이번 달 1일 ~ 오늘
- **기간 변경**: `showDateRangePicker` → 즉시 재로드

순 근무시간 계산 (`_calculateNetMinutes`)

```
총 체류시간 = clockOut − clockIn (분)

휴게시간 = ① breakEnd − breakStart  (실제 기록 우선)
          ② worker.breakMinutes     (기록 없으면 계약상 기본 휴게 자동 차감)

순 근무시간 = max(0, 총 체류시간 − 휴게시간)
```

상황	처리
breakStart/End 모두 기록됨	실제 기록된 휴게 시간 차감
break 기록 없음 + `worker.breakMinutes > 0`	계약상 기본 휴게 자동 차감 + 비고에 "기본휴게 NN분 차감" 표시
`clockOut == null`	0분 (미퇴근) 처리, UI에 "미퇴근" 배지 표시

PDF 구성 (`_buildPdf`)

A4 `MultiPage` 포맷으로 즉석 생성됩니다:

섹션	내용
헤더	"출퇴근 기록부" + 조회 기간
정보 박스	사업장명, 대표자 (Hive StoreInfo), 직원 성명, 생년월일
기록 테이블	날짜 · 요일 · 출근 · 퇴근 · 휴게시작 · 휴게종료 · 비고 · 근무시간 (8열)
비고 자동 기입	`Unplanned` → "비예정", `isSpecialOvertime` → "특별연장", 기본휴게 차감 표시
합계 행	전체 기간 총 근무시간 합산
푸터	"본 출퇴근 기록부는 {사업장명}의 근무 기록을 자동 생성한 서류입니다."

출력 방식

```
Printing.sharePdf(filename: '출퇴근기록부_{직원이름}_{YYYYMM}.pdf')
```

iOS/Android **공유 시트**를 통해 인쇄, 파일 저장, 카카오톡 전송 등 다양한 방식으로 내보낼 수 있습니다.

UI 요약 카드 (화면 상단)

항목	값
총 출근일	`clockOut != null`인 레코드 수
총 근무시간	전체 순 근무시간 합산
미퇴근	`clockOut == null`인 레코드 수

> ***핵심 특징***: *① Firestore 실시간 출퇴근 데이터 → PDF 자동 변환 (별도 입력 불필요), ② 휴게시간 이중 처리 (실제 기록 우선 → 계약 기본값 fallback), ③ 기간 자유 선택, ④ 비고 사유 자동 생성*

7.4. 임금 계약 변경서(임금변경합의서) 전자 작성 및 급여 엔진 자동 연동

시급 또는 월급이 변동될 때, 노동법상 서면으로 변경 합의를 체결하고 이를 급여 계산 엔진에 즉시 반영하는 전자 워크플로우입니다.

7.4.1. 법적 근거 및 작성 의무

- **근로기준법 제17조 (근로조건의 명시)**: 임금, 근로시간 등 주요 근로조건이 변경되는 경우 사용자는 근로자에게 이를 서면으로 명시·교부해야 합니다.
- **판례 (대법원 2019다244122 등)**: 구두 합의만으로는 임금 변경의 적법성 입증이 어렵고, 분쟁 시 서면 증빙이 없으면 사용자에게 불리하게 작용합니다.
- AlbaPay의 임금변경합의서는 이러한 법적 리스크를 줄이기 위해 **전자서명·PDF 렌더링·메타데이터 부인방지** 기능을 근로계약서와 동일한 수준으로 내장하고 있습니다.

7.4.2. 전자 작성 워크플로우

1. **작성 경로**: `노무서류 관리 → 직원 선택 → 새 서류 작성 → 임금 계약 변경서 선택`
2. **구조화 입력 UI**: 변경 전/후 시급(또는 월급), 적용 시작일, 변경 사유 등을 전용 폼(Form)에 입력합니다. 텍스트 수동 입력이 아닌 구조화된 필드를 통해 기재 오류 가능성을 줄여줍니다.
3. **PDF 자동 렌더링**: 입력된 데이터가 노동법 표준 양식(변경 합의서)에 자동 타이핑되어 인쇄/보관용 PDF로 즉시 생성됩니다.

4. **전자 서명 및 부인방지**: 7.2항과 동일한 메타데이터 기반 서명 체계(타임스탬프, IP, 기기ID)가 적용되어, 서명 시점·장소·서명자에 대한 기술적 입증력을 높여줍니다.
5. **교부 및 영구보관**: 서명 완료 즉시 사장님 앱과 알바생 앱 양쪽에 동기화되어 교부 의무를 충족합니다.

7.4.3. 급여 계산 엔진 자동 연동 (wageHistoryJson 시급 변경 이력 추적)

임금변경합의서의 핵심은 단순 문서 발급이 아니라, **변경된 임금이 급여 계산 엔진에 즉시·정확하게 반영**되는 것입니다.

- **관련 변수**: `wageHistoryJson` (JSON 배열), `_getHourlyWageForDate()` (엔진 내부 함수)
- **구현 내용**:
  - **시급 변경 이력 자동 기록**: 임금변경합의서가 교부되는 즉시, 해당 직원의 `wageHistoryJson` 필드에 `[{effectiveDate: "2026-05-06", hourlyWage: 11000}]` 형태로 변경 이력이 **서버(Firestore)와 로컬(Hive) 양쪽에 동시 기록**됩니다.
  - **이중 저장소 동기화 (Dual-Datastore Sync)**: 서버만 업데이트하고 로컬 캐시를 갱신하지 않으면, 이후 다른 화면에서의 저장 작업이 로컬의 빈 이력으로 서버를 덮어쓰는 데이터 유실 위험이 있습니다. AlbaPay 엔진은 이를 방지하기 위해 교부 시점에 반드시 **서버(Firestore `update`)와 로컬(Hive `save`)**을 원자적으로 동기화합니다.
  - **일자별 시급 분할 정산 (Prorated Wage Calculation)**: 급여 계산 엔진은 정산 기간 내 각 출근일마다 `_getHourlyWageForDate()` 함수를 호출하여, `wageHistoryJson`에 기록된 시간순 이력을 조회합니다. 해당 출근일 기준으로 가장 최근에 적용되는 시급을 자동 선택하여 적용합니다.
    - **예시**: 5월 정산 기간에 5/6일부로 시급이 10,320원 → 11,000원으로 변경된 경우
      - 5/1~5/5 출근분 (16시간): `16시간 × 10,320원 = 165,120원`
      - 5/6~5/15 출근분 (24시간): `24시간 × 11,000원 = 264,000원`
      - **기본급 합계**: `165,120 + 264,000 = 429,120원`
    - 이를 통해 월 중간에 시급이 변경되더라도 변경 전/후 각각의 시급이 해당 기간의 출근일에만 정확하게 적용되어, 과다/과소 지급 방지를 고려하여 정산됩니다.
  - **주휴수당 연동**: 시급 변경 시 주휴수당 산정 기준 시급은 **변경 후 시급(최신 시급)**을 기준으로 자동 산출됩니다. 이는 주휴수당이 '소정근로의 대가'로서 변경된 근로조건을 즉시 반영해야 한다는 노무 원칙에 따른 것입니다.

7.4.4. 급여 현황 및 급여 리포트 완전 동기화

- **단일 엔진 원칙**: 급여 현황(대시보드) 화면과 급여 리포트(상세 명세서) 화면은 동일한 `PayrollCalculator.calculate()` 엔진을 호출하여 계산합니다. 따라서 어떤 화면에서 보더라도 동일한 금액이 산출됩니다.
- **시급 분할 내역 UI 표시**: 시급 변경 이력이 있는 직원의 경우, 급여 현황 카드와 급여 리포트 상세 화면 양쪽 모두에 다음과 같은 분할 산출 내역이 표시됩니다:

```
기본급: 40시간 (일자별 변경 시급 적용) = 429,120원
↳ 16시간 × 10,320원 = 165,120원
↳ 24시간 × 11,000원 = 264,000원
```

- **수동 시급 수정 시 이력 보존**: 급여 현황 화면의 '시급 수정' 기능을 통해 시급을 변경하는 경우에도 `wageHistoryJson`이 자동으로 갱신되어, 임금변경합의서를 통한 변경과 동일한 수준의 이력 추적이 유지됩니다. 이력이 유실된 상태에서는 '변경 전 시급' 입력을 통한 이력 복구 기능이 자동으로 활성화됩니다.

7.5. 연차유급휴가 사용촉진 통보서 전자 작성 및 교부

연차 사용촉진제도(6.1.1절 참조)의 법적 이행을 위한 **서면 통보서**를 앱 내에서 작성하고, 알바앱을 통해 전자적으로 교부하는 워크플로우입니다.

7.5.1. 문서 유형 및 법적 구조

문서 유형	DocumentType	근거	내용
1차 촉진 통보서	`leave_promotion_first`	제61조 1항/2항	미사용 연차 내역 통보 + 사용 시기 지정 요청 (직원 10일 이내 회신)
2차 촉진 통보서	`leave_promotion_second`	제61조 1항/2항	직원 미회신 시 사장님이 사용 날짜를 직접 지정하여 서면 통보

- **별도 서류 타입 분리**: 1차와 2차 촉진 통보서를 독립된 `DocumentType`으로 관리하여, Firestore 문서 이력 추적·감사 대응에 유리하도록 설계되어 있습니다.
- 각 통보서에는 연차 배치 발생일·소멸일·미사용 일수·법적 근거(정기/1년 미만 구분)·촉진 기한 유형이 자동으로 기재됩니다.

7.5.2. 전자 작성 워크플로우

1. **작성 경로**: `노무서류 관리 → 직원 선택 → 새 서류 작성 → 연차 사용촉진 1차 통보서 (또는 2차)` 선택
2. **촉진 대상 자동 로드**: 해당 직원의 `leavePromotionLogsJson`에서 촉진 대상 연차 배치가 자동으로 로드됩니다.

    - 1차 통보 시: `pending` 상태인 배치만 표시
    - 2차 통보 시: `first_sent` 또는 `awaiting_plan` 상태인 배치만 표시
3. **배치 선택**: 촉진 대상 배치 중 통보할 대상을 선택합니다. 각 배치별로 연차 유형(정기/1년 미만), 미사용 일수, 소멸일, 1차/2차 촉진 기한이 명시됩니다.
4. **2차 통보 시 날짜 지정**: 2차 촉진의 경우, 사장님이 직원의 연차 사용 날짜를 DatePicker로 직접 지정합니다. 소멸일 이전의 날짜만 선택 가능합니다.
5. **서명 및 교부**: 서명 패드를 통해 사장님 서명 → PDF 자동 생성(`generateLeavePromotionNotice`) → Firebase Storage 업로드 → Firestore 문서 상태 `sent` 로 전환 → 알바앱으로 즉시 교부

7.5.3. 촉진 로그 자동 업데이트

통보서가 교부되는 즉시, 직원의 `leavePromotionLogsJson` 이 자동으로 업데이트됩니다:

- **1차 통보 완료 시**: `firstNoticeDate` = 교부 일시, `firstNoticeDocId` = 문서 ID, `status` = `first_sent`
- **2차 통보 완료 시**: `secondNoticeDate` = 교부 일시, `secondNoticeDocId` = 문서 ID, `designatedDates` = 지정 날짜 목록, `status` = `completed`

이 데이터는 **Hive(로컬)와 Firestore(서버) 양쪽에 동시 저장**되어, 급여 명세서의 사용촉진 현황 UI에 즉시 반영됩니다. 촉진 완료(`completed`) 시 해당 배치의 소멸 연차 수당이 자동으로 면제 처리됩니다.

7.5.4. PDF 통보서 구성 요소

생성되는 PDF에는 다음 정보가 포함됩니다:

구성 요소	내용
제목	"연차유급휴가 사용촉진 통보서 (제1차/제2차)"
법적 근거	근로기준법 제61조 제1항 또는 제2항 (자동 분기)
수신·발신	근로자 성명 / 사업장명·대표자
미사용 연차 테이블	연차 유형, 발생일, 소멸 예정일, 미사용 일수, 촉진 기한 유형
지정 날짜 (2차만)	사장님이 지정한 사용 날짜 목록
서명란	사용자(대표) / 근로자(수령 확인)
감사 메타데이터	생성 시각, 무결성 해시(SHA-256), 앱 자동 생성 표시

---

8. 알바생 전용 앱(Alba App) 사용자 워크플로우 명세

알바생이 앱을 설치하고 로그인하여 출퇴근 및 공지사항을 확인하는 전체 워크플로우와 내부 처리 로직입니다.

8.1. 간편 로그인 및 기기 연동 (Deep Link & Code)

알바생은 복잡한 회원가입 절차 없이 사장님이 발급한 초대 코드를 통해 즉시 앱에 연동됩니다.

- **초대 링크 (Deep Link):** 문자로 받은 초대 링크를 클릭하면 앱이 자동 실행되며, `app_links` 를 통해 파라미터를 가로채 즉각 연동 처리됩니다.
- **알바 코드 직접 입력:** 앱 초기 화면에서 6자리 고유 알바 코드(`workerId`)를 직접 입력하여 연동할 수도 있습니다.
- **상태 유지:** 연동된 정보는 로컬 기기에 영구 저장되어 앱 재실행 시 자동 로그인됩니다.

8.2. 출퇴근 기록 엔진 (스마트 분쟁 방어 로직)

출퇴근 행위는 단순한 시간 기록을 넘어, **노무 분쟁 방지**를 위한 여러 예외 처리(Edge Cases) 로직을 거칩니다.

- **출근 처리 (`_clockIn`):**
    - 버튼 터치 즉시 Firestore에 `Attendance` 문서를 생성하여 기기 조작을 방지하고 서버 타임스탬프를 기준으로 출근 기록을 확정합니다.
    - 출근과 동시에 백그라운드에서는 사장님이 설정한 **예정 퇴근 시간(Scheduled End)**을 불러와 캐싱합니다.
- **퇴근 처리 및 조기퇴근/연장근로 방어 (`_clockOut`):**

- **조기 퇴근 방어:** 예정된 퇴근 시간보다 일찍(유예 시간 초과) 퇴근 버튼을 누를 경우, 시스템(`isEarlyClockOut`)은 이를 감지하고 다이얼로그를 띄워 **"조기 퇴근 사유(예: 개인 사정, 병원 등)"**를 의무적으로 입력하도록 강제합니다.
- **연장 근로 승인:** 반대로 퇴근 시간을 넘겨 퇴근할 경우, **"연장 근무를 신청할 것인지, 단순히 퇴근 버튼을 늦게 누른 것인지"** 본인이 직접 선택하게 합니다. 연장 근무 신청 시 사유를 입력받아 추후 수당 지급의 법적 근거로 남깁니다.
- **UI 안정성 보호:** 조기 퇴근 등 비동기 다이얼로그 호출 과정에서 Flutter 위젯 트리가 꼬이지 않도록 위젯 생명주기를 최적화하여 튕김(Crash) 현상을 원천 차단합니다.

8.3. 공지사항 및 업무 지시 확인 (Read Receipt Tracking)

단순한 게시판 형태를 넘어, 법적 고지 의무 및 업무 지시 확인을 기록하기 위한 **수신 확인 트래킹** 로직이 포함되어 있습니다.

- **대시보드 실시간 반영:** `stores/{storeId}/notices` 컬렉션을 실시간으로 구독하여 최신순으로 3개의 공지를 대시보드에 노출합니다.
- **읽음 상태 추적 (`reads` Subcollection):**
  - 각 공지사항 문서 하위에 `reads/{workerId}` 서브 컬렉션을 두어, 알바생이 해당 공지를 터치하여 열람하는 순간 Firestore에 타임스탬프를 남깁니다.
  - 이를 통해 대시보드에서 **아직 읽지 않은 공지에만 빨간 점(N 뱃지)**을 띄워 열람을 유도하며, 사장님은 어떤 직원이 공지를 확인했는지 추적(노무 이력 관리)할 수 있습니다.

9. [부록] 체험 모드(Demo) 노무 검증 시나리오 가이드

AlbaPay 엔진은 체험 모드(가상 데이터)와 실서비스 환경에서 동일한 계산 로직을 적용하는 **"로직 패리티(Logic Parity)"** 원칙을 따릅니다. 체험용으로 임의 생성된 가상 직원들의 출퇴근 기록(과거 400일치)에는 엔진의 계산 다양한 상황 대응을 시뮬레이션하기 위한 **엣지 케이스 시나리오가 포함**되어 있습니다. 앱 로그인 후, 직원들의 지난달 **'급여 명세서 상세'**를 열람하시면 아래 법적 쟁점들을 직접 검증하실 수 있습니다.

9.1. 시나리오 A: 100% 만근 및 연차 사용촉진 (대상자: `가상 김점장`)

- **주입된 데이터**: 결근 및 연차 사용 기록이 전혀 없는 100% 개근 데이터 (400일 연속).
- **검증 포인트**:
  1. 주휴수당이 단 한 주도 삭감되지 않고 100% 정상 지급되는지 확인.
  2. 1년 근속 후 연차(15일)가 정상적으로 전액 생성되는지 확인.
  3. 명세서 하단의 **[연차 사용촉진 현황]** UI에 "⏳ 촉진 미이행" 또는 1차/2차 통보 기한이 정상적으로 표출되는지 검증 (근로기준법 제61조 연차 사용촉진제도 적용 여부 확인).

9.2. 시나리오 B: 연차 사용에 따른 주휴수당 보존 (대상자: `가상 이주간`)

- **주입된 데이터**: **4월 14일(화)~16일(목)** 3일간의 '연차(Leave)' 출퇴근 기록 주입. 해당 주(4/13~17) 4일 근무 중 3일을 연차로 사용한 상태.
- **검증 포인트**:
  1. 4월 3주차에 출근하지 않고 연차를 3일 사용했음에도, 대법원 판례에 따라 **해당 주차의 주휴수당이 삭감되지 않고 100% 정상 지급**되는지 확인.
  2. 연차 사용으로 인해 4월의 "1개월 개근 시 1일 부여"되는 신입 연차 요건이나, 1년간 80% 출근율 산정에 불이익(결근 처리)이 발생하지 않는지 검증.

9.3. 시나리오 C: 무단 결근에 따른 즉각적인 주휴 삭감 (대상자: `가상 박오전`)

- **주입된 데이터**: **4월 7일(화)~8일(수)** 이틀간 출퇴근 기록 미생성 (결근 처리). 해당 주(4/6~9) 4일 근무 중 2일을 무단 결근.
- **검증 포인트**:
  1. 4월 2주차에 결근이 감지되어 **해당 주차의 주휴수당이 0원으로 삭감**되는지 명세서에서 확인.
  2. 결근이 포함된 4월의 **'1개월 만근 시 1일 부여' 연차가 자동으로 미발생 처리**되어 엔진의 데이터 기반 근태 평가 로직이 정상 작동하는지 검증.
  3. 명세서 최상단에 **"⚠️ 결근 감지 (주휴수당 자동 삭감)" 적색 경고 카드**가 노출되는지 확인. 이를 통해 매장 사정(휴업)으로 쉰 경우에 대한 사장님의 수동 지급 의무(노동법 리스크)를 앱이 적절히 고지하고 방어하는지 검증.

8.3.1. 검증 결과 — 현재 구현 상황

검증 항목	기대 동작	실제 동작	결과
① 결근 감지	`weeklyHolidayBlockedByAbsence = true`	✅ 정상 감지	**PASS**
② 주휴수당 자동 삭감	해당 주차 주휴수당 0원	⚠️ 주휴수당 57,600원 **정상 지급**	**설계 의도**
③ 적색 경고 카드	🔴 적색 경고	🔵 파란색 정보 카드 표시	**설계 의도**

8.3.2. 카드 색상 분기 로직 (Safe-by-Default 설계)

경고 카드의 색상과 메시지는 `approved` 변수에 의해 결정됩니다:

```
// payroll_report_page.dart L595
final approved = _manualWeeklyHolidayApproval[staff.id] ?? staff.weeklyHolidayPay;

// staff.weeklyHolidayPay 기본값 = true (Worker 모델)
// → 사장님이 명시적으로 끄기 전까지 "수동 승인됨" 상태
```

approved 값	카드 색상	제목	주휴수당
true (기본)	🔵 파란색	"결근 감지 (수동 승인됨)"	정상 지급
false	🔴 적색	"결근 감지 (주휴수당 자동 삭감)"	0원 삭감

8.3.3. 설계 의도 — "안전 우선(Safe-by-Default)" 철학

결근이 감지되어도 **기본적으로 주휴수당을 지급**하고 사장님에게 알리는 방식을 채택한 이유:

1. **자동 삭감의 위험**: 매장 사정(휴업·경영상 사유)으로 쉬게 한 경우까지 자동 삭감하면 **임금체불**이 됨
2. **사장님 판단 우선**: 결근 사유(무단결근 vs 사장님 지시 휴무)는 앱이 판별할 수 없으므로, 사장님이 명시적으로 `weeklyHolidayPay = false` 로 설정할 때만 삭감
3. **노동법 방어**: 근로기준법 제46조(휴업수당) — 사용자 귀책 사유로 쉬게 한 경우 평균임금 70% 이상 지급 의무

> ***결론**: 현재 박오전의 파란색 카드는 엔진이 결근을 감지하되, 사장님이 아직 삭감 결정을 내리지 않은 상태를 반영합니다. 사장님이 직원 설정에서 주휴수당 토글을 끄면 즉시 🔴 적색 경고 카드 + 주휴수당 0원 삭감으로 전환됩니다.*

9.4. 시나리오 D: 초단시간 근로자 처리 검증 (주 14시간 고정 근무, 400일 퇴사)

- **주입된 데이터**: 주 2일, 하루 7시간 근무 (주 14시간 고정) 조건으로 400일(약 1년 1개월)간 100% 만근 후 퇴사 처리.
- **검증 포인트**:
  1. **주휴수당 0원 고정**: 400일 내내 만근했음에도, 1주 15시간 미만이므로 `isWeeklyHolidayEligible` 가 차단되어 주휴수당이 단 한 번도 발생하지 않는지 확인.
  2. **연차/연차수당 0원 차단**: `AnnualLeaveCalculator` 가 "초단시간 근로자: 연차 발생 대상 아님" 사유와 함께 신입 연차(11개) 및 정기 연차(15개) 발생을 차단하여, 퇴사 시 지급해야 할 미사용 연차수당이 0원인지 검증.
  3. **퇴직금 제외**: 1년 이상 근무했음에도 4주 평균 15시간 미만이므로 퇴직금 산정 대상에서 배제되는지 확인.
  4. **근로자의 날(5/1) 유급휴일 특례 작동**: 주휴/연차/퇴직금이 모두 없더라도, 법정 유급휴일인 근로자의 날에는 단시간 비례 공식 `(14시간 ÷ 40시간) × 8시간 = 2.8시간` 에 해당하는 수당이 산출되어 명세서에 합산되는지 확인.

9.5. 시나리오 E: 격주 변동 스케줄의 4주 평균 산출 방어 (1주차 14시간, 2주차 16시간 반복)

- **주입된 데이터**: 1주 차는 14시간, 2주 차는 16시간 근무를 반복하는 스케줄로 400일간 만근 후 퇴사 처리.
- **검증 포인트**:
  1. **4주 평균 15시간 판정 작동**: 4주간 총 근로시간 60시간(14+16+14+16)을 4주로 나눈 **평균 15시간**을 엔진이 인지하여, 초단시간 근로자 방어막을 해제하는지 확인.
  2. **14시간 근무 주간의 주휴수당 지급**: 사장님이 임의로 "14시간 일한 주는 주휴수당이 없다"고 착각하기 쉬우나, 엔진이 4주 평균(15시간)을 기준으로 매주 `(15시간 ÷ 40시간) × 8시간 = 3시간` 분의 주휴수당을 14시간 일한 주간에도 빠짐없이 100% 정상 지급하여 '임금체불 리스크'를 자동 방어하는지 검증.
  3. **연차 정상 발생**: `AnnualLeaveCalculator` 가 정상 가동되어 총 26개(1년 차 11개 + 2년 차 15개)의 연차가 부여되며, 퇴사 시 26개 × 3시간 분의 미사용 연차수당이 정상 정산되는지 확인.
  4. **퇴직금 대상 편입**: 4주 평균 15시간 이상 & 1년 이상 근무 조건을 충족하여 퇴직금 지급 대상으로 자동 분류되는지 검증.

9.6. 시나리오 F: 5인 이상/미만 사업장 전환에 따른 가산수당 분리 검증 (14:00~24:00, 휴게 1시간)

- **주입된 데이터**: 매일 14:00~24:00 (총 10시간 체류, 밤 10시 이전 1시간 휴게로 순수 9시간 근무) 스케줄을 배정.
- **검증 포인트**: 매장 설정에서 '상시 근로자 수 5인 이상' 토글 스위치를 껐다 켰을 때, 명세서 금액이 즉각적으로 변동되는지 확인.
  1. **[5인 이상] 설정 시 (가산 적용)**:
     - 연장근로 가산(8시간 초과 1시간): `1시간 × 0.5배 = 0.5시간 치 시급 추가`
     - 야간근로 가산(22:00~24:00 2시간): `2시간 × 0.5배 = 1.0시간 치 시급 추가`

    - **결과**: 하루 기본급(9시간) + 가산수당(1.5시간) = **하루 10.5시간 분의 시급**이 지급되는지 명세서 상세 내역(연장/야간 항목)에서 확인.
  2. **[5인 미만] 설정 시 (가산 미적용)**:
    - 명세서 내 연장근로수당 및 야간근로수당이 즉시 0원으로 락(Lock)이 걸림.
    - **결과**: 오로지 순수 일한 시간인 **하루 9.0시간 분의 시급**만 정직하게 계산되어, 5인 이상일 때와 비교해 하루 1.5시간 치의 시급 차이가 반영되는지 검증.

9.7. 시나리오 G: 연차 사용촉진제도 적용 및 수당 면제 검증 (대상자: `가상 김점장`)

- **주입된 데이터**: 1년 미만 입사자의 정기 연차(월별 발생분 포함)에 대해 사장님이 1차/2차 사용촉진 서면 통보를 모두 완료한 상태(`status: completed`)를 시뮬레이션으로 주입.
- **검증 포인트**:
  1. **촉진 완료 뱃지 노출**: 급여 리포트 화면의 연차 상태 UI가 "⏳ 촉진 미이행"에서 **"✅ 촉진 완료"** 뱃지로 변경되는지 확인 (UI 오버플로우 없이 정상 렌더링).
  2. **수당 면제(차감) 로직 작동**: 촉진제 절차를 적법하게 완료했음에도 직원이 사용하지 않고 소멸된 연차에 대해, 엔진의 `promotionExemptPayoutAmount` 가 작동하여 당월 **미사용 연차수당 지급액에서 해당 금액만큼 자동 차감(면제) 표기**가 이루어지는지 검증.
  3. **데이터 무결성 유지**: 엔진으로 전달되는 `workerData` 에 파싱된 `promotionLogs` 가 누락 없이 주입되어, 겉보기 UI 표출뿐 아니라 실제 백엔드 정산 엔진까지 정상적으로 연동되는지 확인.

9.8. [실습] 앱에서 직접 조작하며 검증하는 방법

> ***대상***: *노무사, QA 담당자, 또는 엔진 정합성을 직접 눈으로 확인하고 싶은 사장님* ***환경***: *체험 모드(가상 데이터) 로그인 후 사장님 앱 (모바일 또는 웹)* ***원칙***: *가상 데이터의 출퇴근 기록을* ***삭제·수정·추가****한 뒤 급여 명세서를 새로고침하면, 엔진이 실시간으로 재계산한 결과를 즉시 비교할 수 있습니다.*

실습 전 준비

1. 앱에 **체험 모드**로 로그인합니다 (초대코드 입력 없이 "체험하기" 버튼).
2. **급여 관리** 탭에서 지난달 정산 기간을 선택합니다.
3. 아래 가상 직원 프로필을 참고하여 대상 직원의 **급여 상세 보기**를 열어 현재 금액을 메모합니다.

가상 직원 프로필 (체험 모드 자동 생성)

ID	이름	시급	근무일	근무시간	주간시간	특이사항
worker_a	**가상 김점장**	월급제(250만)	월~금	09:00~18:00	40h	전일제, 연차 미사용(촉진 대상), 서류 서명 완료
worker_b	**가상 이주간**	10,500원	월~목	10:00~18:00	28h	유급휴게, 연차 3일 사용, **15~17일 전 연차 3일 연속 시나리오**
worker_c	**가상 박오전**	12,000원	월~목	06:00~13:00	24h	오전 조기 출근, 연차 8일 사용, **10~11일 전 무단결근 2일 시나리오**
worker_d	**가상 최오후**	10,320원	월~목	14:00~21:00	24h	오후~저녁, 연차 2일 사용
worker_e	**가상 강야간**	10,320원	월~목	22:00~06:00	28h	**야간근무(22~06시)**, 야간수당 대상, 연차 1일 사용
worker_f	**가상 정금요**	10,320원	금	12:00~21:00	8h	주1일, **주 15시간 미만(초단시간)**, 연차 미발생
worker_g	**가상 조토요**	11,000원	토	12:00~21:00	8h	주1일, 초단시간
worker_h	**가상 윤일요**	10,320원	일	12:00~21:00	8h	주1일, 초단시간

> 📌 ***자동 시나리오**: 체험 모드 진입 시 `가상 박오전` 에게는 10~~11일 전 **무단결근 2일이,** `가상 이주간` 에게는 15~~17일 전 **연차 3일**이 자동 주입되어 있습니다. 이 시나리오가 급여에 반영된 상태에서 실습을 시작합니다.*

실습 1: 만근자의 출근 기록 삭제 → 결근 효과 검증 (대상: `이주간`)

단계	조작 방법	기대 결과
①	급여 상세 → **주차별 출퇴근 내역** 확인. 아무 주차나 선택하여 **특정 평일(예: 5/5 월요일)의 출근 기록**을 메모합니다.	해당 주차 주휴수당이 정상 지급(≈58,800원) 상태
②	**출퇴근 관리** 메뉴 → 이주간 → 5/5(월) 출근 기록을 **삭제**합니다.	출근 기록이 사라짐
③	**급여 관리** 탭으로 돌아가 지난달 명세서를 **새로고침**(당겨서 새로고침 또는 재진입)합니다.	엔진이 재계산 시작
④	변경된 명세서를 확인합니다.	아래 3가지 변화 확인

기대 변화:

항목	삭제 전 (만근)	삭제 후 (결근 1일)	차이
기본급	5/5 포함	5/5 **빠짐** (1일분 감소)	**-73,500원**
해당 주 주휴수당	58,800원	⚠️ 파란색 카드 (결근 감지)	삭감 대기 상태
총급여	원래 금액	원래 금액 - 73,500원	**기본급 1일분 차감**

> 💡 ***핵심 관찰**: 결근 감지 시 주휴수당은 **즉시 삭감되지 않고** 파란색 안내 카드로 전환됩니다 (§8.3.3 Safe-by-Default). 사장님이 토글을 끄면 그때 0원으로 삭감됩니다.*

복원: 출퇴근 관리 → 이주간 → 5/5에 **수동 출퇴근 추가** (10:00~18:00) → 명세서 새로고침 → 원래 금액으로 복원 확인.

실습 2: 조퇴 기록으로 변경 → 주휴수당 유지 확인 (대상: `이주간`)

단계	조작 방법	기대 결과
①	출퇴근 관리 → 이주간 → **5/7(수) 퇴근 시간을 18:00 → 14:00으로 수정**	조퇴(3시간만 근무)
②	급여 명세서 새로고침	엔진 재계산

기대 결과:

항목	수정 전	수정 후 (조퇴)	변화
5/7 기본급	7h × 10,500 = 73,500원	3h × 10,500 = **31,500원**	-42,000원
해당 주 주휴수당	58,800원	58,800원 **그대로**	**변화 없음** ✅

> 💡 ***핵심 관찰**: 조퇴는 **출근한 것이므로 결근이 아닙니다.** 주휴수당은 "소정근로일에 출근했는가"만 보기 때문에 3시간을 일했든 7시간을 일했든 주휴수당이 유지됩니다. (§5.1 지각/조퇴 보호)*

실습 3: 5인 이상 ↔ 5인 미만 토글 전환 → 가산수당 즉시 변동 (대상: 전체)

단계	조작 방법	기대 결과
①	**매장 설정** → '상시 근로자 수 5인 이상' 토글을 **끔(OFF)**	5인 미만 전환
②	급여 관리 → 야간 근무자의 명세서 확인	가산수당 항목 변동

기대 결과 (야간 근무자 1일 기준):

항목	5인 이상 (ON)	5인 미만 (OFF)	차이
연장근로수당	1h × 10,320 × 0.5 = 5,160원	**0원**	-5,160원
야간근로수당	2h × 10,320 × 0.5 = 10,320원	**0원**	-10,320원
하루 합계 차이	—	—	**-15,480원/일**

> 💡 ***핵심 관찰***: *토글 하나로 가산수당이 즉시 0원 ↔ 정상으로 왔다갔다하는 것을 눈으로 확인할 수 있습니다. 이것이 5인 이상/미만 분기의 실질적 금액 차이입니다.*

실습 4: 연차 등록 → 주휴수당 보존 + 기본급 0원 확인 (대상: `이주간`)

단계	조작 방법	기대 결과
①	**스케줄 관리** → 이주간 → 지난달의 아무 평일을 **연차(Annual Leave)로 등록**	해당일 연차 처리
②	급여 명세서 새로고침	엔진 재계산

기대 결과:

항목	연차 등록 전	연차 등록 후
해당일 기본급	73,500원	**0원** (clockIn=clockOut → 근로시간 0분)
해당 주 주휴수당	58,800원	58,800원 **유지** ✅
연차 잔여일	N일	**N-1일**

> 💡 ***핵심 관찰***: *연차 사용일은 기본급은 0원이지만, 별도 연차유급수당(`annualLeaveUsedPay`)으로 **1일 소정근로시간 × 시급**이 지급됩니다. 또한 **출근으로 간주**되어 주휴수당이 유지됩니다. 이것이 결근과 연차의 결정적 차이입니다. (근로기준법 제60조 제6항)*

실습 5: 시급 수정 → 분할 계산 확인 (대상: `최오후`)

단계	조작 방법	기대 결과
①	**직원 관리** → 최오후 → **시급 수정**: 10,320원 → **11,000원** (변경일: 지난달 16일)	월 중간 시급 변경
②	급여 명세서 새로고침	시급 분할 계산

기대 결과:

```
기본급: 00시간 (일자별 변경 시급 적용) = 000,000원
 ↳ 16일 이전: XX시간 × 10,320원 = XXX,XXX원
 ↳ 16일 이후: YY시간 × 11,000원 = YYY,YYY원
```

> 💡 ***핵심 관찰***: *시급 변경 이력(`wageHistoryJson`)이 자동 기록되어, 변경일을 기준으로 이전/이후 근무시간이 각각의 시급으로 분리 계산됩니다. (§7.4.3)*

실습 6: 수습기간 설정 → 90% 감액 확인 (대상: `최오후`)

단계	조작 방법	기대 결과
①	**직원 관리** → 최오후 → 수습기간 **ON**, 3개월 설정	수습 활성화
②	급여 명세서 새로고침	기본급 10% 감소

기대 결과:

항목	수습 OFF	수습 ON (90%)

적용 시급	10,320원	floor(10,320 × 0.9) = **9,288원**
6시간 기본급	61,920원	**55,728원**

> ⚠️ ***주의**: 수습 시급(9,288원)이 당해 연도 최저임금(10,320원) 미만이면 앱이 경고를 표시합니다. 이 경우 수습 감액이 위법할 수 있으므로 노무사 상담을 권장합니다.*

실습 7: 전일제 직원 퇴사 → 퇴직금 정산서 확인 (대상: `김점장`)

> ***전제조건**: 김점장은 가상 데이터 생성 시점으로부터 약 400일 전에 입사한 전일제(주 40시간) 직원이며, 시급 10,320원으로 설정되어 있습니다.*

단계	조작 방법	기대 결과
①	**직원 관리** → 김점장 → 오른쪽 🚪 **퇴사 아이콘** 터치	퇴사 일자 선택 달력 표시
②	퇴사일을 **오늘 날짜**로 선택 → **"정산하기"** 터치	퇴사 정산 리포트 화면 진입
③	정산 리포트에서 아래 항목들을 확인합니다	엔진이 자동 산출한 정산 내역

확인해야 할 핵심 수치:

항목	확인 포인트	산식 근거
재직일수	≥ 365일 (약 400일)	입사일~퇴사일 차이
퇴직금 자격	"1년 이상 근무" 표시 ✅	재직일수 ≥ 365 AND 주 15시간 이상
1일 평균임금	자동 산출 (역월 3개월)	퇴사 전 3개월 총 임금 ÷ 역일수(89~92일)
퇴직금	> 0원 (약 250만 원 이상)	`1일 평균임금 × 30 × (재직일수 ÷ 365)`
잔여 연차수당	발생 연차 기반 정산	`(발생 연차 − 사용 연차) × 8h × 시급`
지급 기한	퇴사일 + 14일	근로기준법 제36조

> 💡 ***핵심 관찰**:*
>
> - ***역월 3개월**: 90일이 아닌 **실제 달력 일수(89~92일)**로 나눕니다. 퇴사일이 5월 말이면 92일, 4월 말이면 89일입니다.*
> - ***통상임금 비교**: 엔진은 `Max(평균임금, 통상임금)` 중 높은 쪽을 선택합니다.*
> - ***PDF 저장**: "정산서 PDF로 저장" 버튼을 눌러 실제 PDF가 생성되는지도 확인하세요.*

⚠️ **복원 주의**: "최종 퇴사 처리"를 누르면 직원이 비활성 상태로 전환됩니다. 체험 모드에서는 다시 체험을 초기화해야 복원됩니다. 금액만 확인하려면 **PDF 저장만 하고 퇴사 처리는 누르지 마세요.**

실습 8: 초단시간 직원 퇴사 → 퇴직금 0원 확인

> ***조건**: 가상 직원 중 주 14시간(주 2일 × 7시간) 근무자가 400일 이상 재직한 경우*

단계	조작 방법	기대 결과
①	직원 관리 → 초단시간 직원 → 퇴사 아이콘	퇴사 일자 달력
②	오늘 날짜 선택 → 정산하기	퇴사 정산 리포트
③	아래 항목 확인	자격 배제 확인

기대 결과:

항목	확인 포인트
재직일수	≥ 365일 (1년 이상 근무)

퇴직금 자격	**"1년 미만 근무(미대상)" 또는 퇴직금 0원**
퇴직금	**0원** ✅
연차수당	**0원** (초단시간은 연차 미발생)

> 💡 ***핵심 관찰***: *400일이나 일했는데도 퇴직금이 0원인 이유는* ***주 15시간 미만(초단시간)*** *이기 때문입니다. 퇴직급여보장법 제4조에 의해 주 15시간 미만 근로자는 재직 기간에 관계없이 퇴직금 대상에서 제외됩니다.*

실습 9: 1년 미만 단기 직원 퇴사 → 연차만 정산

> ***조건***: *입사일을 최근 6개월 전으로 수정하거나, 이미 6개월 미만 재직 중인 직원*

단계	조작 방법	기대 결과
①	직원 관리 → 아무 직원 → 수정 → **입사일을 6개월 전으로 변경**	재직 기간 단축
②	해당 직원 → 퇴사 아이콘 → 오늘 날짜 → 정산하기	퇴사 정산 리포트

기대 결과:

항목	값	비고
재직일수	≈ 180일	< 365일
퇴직금 자격	**미대상** ❌	1년 미만
퇴직금	**0원**	자격 미달
잔여 연차	약 5~6일	월별 1개씩 발생 (5인 이상 시)
연차수당	**>0원**	잔여 연차 × 8h × 시급

> 💡 ***핵심 관찰***: *퇴직금은 0원이지만* ***미사용 연차수당****은 정상 정산됩니다. 1년 미만 근로자도 매월 만근 시 1개의 연차가 발생하며, 퇴사 시 미사용분은 수당으로 지급해야 합니다.*

실습 체크리스트 (검증 완료 기록용)

#	실습	조작	예상 동작	확인
1	결근 효과	출근 기록 삭제	기본급 1일분 차감 + 결근 카드	☐
2	조퇴 보호	퇴근 시간 앞당김	주휴수당 유지	☐
3	5인 분기	매장 설정 토글	가산수당 0원 ↔ 정상	☐
4	연차 보호	연차 등록	기본급 0원 + 주휴 유지	☐
5	시급 분할	월 중간 시급 변경	분할 내역 표시	☐
6	수습 감액	수습 ON	시급 × 0.9 적용	☐
7	전일제 퇴직금	김점장 퇴사 처리	퇴직금 > 0 + 역월 3개월 + PDF	☐
8	초단시간 퇴직금	주14h 직원 퇴사	퇴직금 0원 (자격 배제)	☐
9	1년 미만 퇴직금	6개월 직원 퇴사	퇴직금 0원 + 연차수당만 정산	☐

> ***검증 완료 후***: *모든 조작은 체험 모드(가상 데이터) 내에서 이루어지므로 실서비스에 영향이 없습니다. 데이터를 초기 상태로 되돌리려면* ***설정 → 로그아웃 → "체험하기" 재진입****하세요. 새로운 익명 계정으로 깨끗한 가상 매장이 자동 생성됩니다.*

9.9. 가상 데이터로 검증 불가능한 영역 — 실기기·실서비스 테스트 필수 항목

> *§8.1~8.8의 모든 검증은 **계산 엔진의 산출 결과**에 초점을 맞추며, 체험 모드(가상 데이터)에서 100% 완결됩니다. 그러나 아래 영역은 **실제 기기 하드웨어, 네트워크 환경, 또는 사장님↔알바생 간 양방향 통신**에 의존하므로 반드시 실기기에서 별도로 검증해야 합니다.*

🔴 **카테고리 A: 반드시 실기기에서만 테스트 가능**

#	기능	의존 대상	가상 데이터로 안 되는 이유	테스트 방법
A-1	**QR/바코드 출퇴근 스캔**	카메라 하드웨어	체험 모드는 출근 기록이 Firestore에 직접 주입되므로 카메라 → QR 인식 → 출근 기록 생성의 **전체 흐름**이 검증되지 않음	실제 스마트폰에서 알바앱 → QR 스캔 → 사장님 앱에서 출근 기록 확인
A-2	**알바앱 ↔ 사장님앱 실시간 연동**	Firestore 실시간 리스너	체험 모드는 사장님 앱 단독 환경. 알바생이 출근 찍으면 사장님 대시보드에 실시간 반영되는 **양방향 동기화**가 검증 불가	기기 2대: 1대는 사장님 앱, 1대는 알바앱. 알바생이 출근 → 사장님 화면에 즉시 반영 확인
A-3	**노무서류 전자서명 + PDF 생성**	터치스크린 서명 → PDF 렌더링 → Firebase Storage 업로드	체험 모드의 서류는 `status: draft/signed`로 사전 주입된 더미 데이터. **실제 서명 패드 → PDF 생성 → 알바앱 교부**의 End-to-End 흐름 미검증	사장님 앱 → 노무서류 관리 → 근로계약서 작성 → 서명 → PDF 생성 → 알바앱에서 수령 확인
A-4	**급여 명세서 PDF 공유/다운로드**	OS 공유 시트, 파일 시스템	체험 모드에서 명세서 금액 검증은 가능하나, **PDF 파일로 내보내기 → 카카오톡/이메일 전송** 흐름은 OS 의존	급여 상세 → 명세서 공유 버튼 → PDF 생성 → 외부 앱으로 전송 성공 확인
A-5	**출퇴근 바코드 인쇄** (사장님 앱)	프린터 연결 (AirPrint 등)	체험 모드에는 인쇄 대상 바코드 이미지는 렌더링되나, 실제 프린터 출력까지는 검증 불가	매장 설정 → 출퇴근 바코드 → 인쇄 → 실제 종이 바코드로 알바앱 스캔 테스트

🟡 **카테고리 B: 가상 데이터로 부분 검증 가능하나 실기기 보완 필요**

#	기능	가상 데이터에서 검증 가능한 부분	실기기에서 추가 검증해야 할 부분
B-1	**Unplanned 출근 승인 흐름**	계산 엔진이 `Unplanned` 상태를 급여에서 제외하는 로직 ✅	알바생이 스케줄 없는 날 QR 스캔 → 사장님 앱에 **승인 요청 알림** 표시 → 승인 버튼 터치 → `UnplannedApproved`로 전환 → 급여 재산정
B-2	**연차 사용촉진 서면 통보**	엔진의 촉진 기한 계산, 면제액 산출, `promotionStatuses` 자동 생성 ✅	사장님 앱 → 촉진 통보서 작성 → 서명 → **PDF 교부 → 알바앱에서 수령** → `leavePromotionLogsJson` Firestore 갱신 확인
B-3	**임금 변경 합의서**	엔진의 `wageHistoryJson` 분할 계산 ✅	사장님 앱 → 임금 변경서 작성 → 양측 서명 → Worker 모델의 `wageHistoryJson` 자동 업데이트 → **변경 전후 시급 분할 명세서** 즉시 반영
B-4	**퇴사 정산서 (Exit Settlement)**	`SeveranceCalculator` 엔진의 산출 결과 (퇴직금, 연차수당, 역월 3개월) ✅	사장님 앱 → 직원 퇴사 처리 → **퇴사 정산서 PDF 자동 생성** → 금액이 엔진 출력과 일치하는지 PDF 레이아웃 + 금액 교차 검증
B-5	**대타(근무 교환) 승인 흐름**	교환 후 출퇴근 기록이 올바른 직원에게 귀속되는 계산 로직 ✅	알바생A ↔ 알바생B 대타 요청 → 사장님 승인 → 해당일 스케줄 교환 → **교환된 스케줄에 맞는 출퇴근 기록 생성** 확인

⚪ **카테고리 C: 현재 미구현 또는 향후 업데이트 시 추가 검증 필요**

#	기능	현재 상태	향후 테스트 필요 시점
C-1	**GPS 반경 기반 출퇴근 제한**	미구현 (향후 로드맵)	GPS 하드웨어 + 위치 권한 + 반경 계산
C-2	**WiFi SSID 기반 출퇴근 인증**	미구현 (향후 로드맵)	특정 WiFi 네트워크 연결 상태 확인
C-3	**푸시 알림 (결근 감지, 촉진 기한 임박)**	미구현 (향후 로드맵)	FCM 토큰 등록 + 백그라운드 알림 수신
C-4	**공공데이터포털 연동 (법정 공휴일 API)**	미구현 (§3.2에 명시)	근로자의 날(5/1) 외 빨간 날 자동 적용

실기기 테스트 체크리스트

> *아래는 실서비스 배포 전 또는 노무 감사 대응 시 반드시 수행해야 할 End-to-End 테스트 목록입니다.*

#	테스트 시나리오	필요 기기	예상 소요	확인
1	알바생 QR 출근 → 사장님 대시보드 실시간 반영	폰 2대	3분	☐
2	알바생 QR 퇴근 → 급여 명세서 자동 반영	폰 2대	3분	☐
3	스케줄 없는 날 출근(Unplanned) → 사장님 승인 → 급여 포함	폰 2대	5분	☐
4	근로계약서 작성 → 양측 서명 → PDF 생성 → 알바앱 교부	폰 2대	10분	☐
5	임금 변경서 작성 → 시급 변경 → 명세서 분할 계산 확인	폰 1대 (사장님)	5분	☐
6	퇴사 처리 → 퇴사 정산서 PDF → 금액 교차 검증	폰 1대 (사장님)	10분	☐
7	출퇴근 바코드 인쇄 → 종이 바코드로 알바앱 스캔	폰 + 프린터	5분	☐
8	급여 명세서 PDF 공유 (카카오톡/이메일)	폰 1대	3분	☐

> ***핵심 원칙**: "**숫자(계산)는 가상 데이터로 증명하고, 흐름(워크플로우)은 실기기로 검증한다.**" 두 영역을 분리하면, 계산 엔진의 산출 로직은 114개 자동화 테스트로 검증하면서도, UI/UX와 하드웨어 연동은 실기기 테스트로 보완할 수 있습니다.*

부록 A. 퇴직금 정산 엔진 — 가상 데이터 계산 결과 시뮬레이션

> ***목적**: 엔진이 출력하는 최종 퇴직금 금액이 "왜 이 숫자인지"를 중간값 전수 추적으로 수치로 시뮬레이션합니다. **테스트 파일**: `packages/shared_logic/test/compliance/severance_pay_proof_test.dart` **실행**: `flutter test test/compliance/severance_pay_proof_test.dart` — 4개 시나리오 전부 통과 ✅*

A-1. 시나리오 A: 전일제(40h) 이론 추산 — 10단계 전수 검증

가상 데이터 설정

항목	값
입사일	2025-03-01
퇴사일	2026-05-31
주간 소정근로	40시간 (전일제)
시급	10,320원
5인 이상	✅
출근기록	없음 (가상직원 → 이론 추산 경로)

10단계 계산 추적

단계	항목	산식	결과	엔진 일치
①	재직일수	2025-03-01 ~ 2026-05-31	**457일** (≥365 ✓)	✅
②	역월 3개월 기간	5/31의 3개월 전 = 2/28 (비윤년)	**92일** (not 90)	✅
③	1일 소정근로시간	40h ≥ 40 → 8.0h	**8.0h**	✅
④	주간 총 유급시간	소정(40) + 주휴(8)	**48.0h**	✅
⑤	3개월 기본급+주휴	48 × (92÷7) × 10,320	**6,510,446원**	✅
⑥	연차수당 산입 (※)	발생 확정 연차 15일 × 8h × 10,320 × (92÷365)	**312,145원**	✅

⑦	3개월 총 임금	⑤ + ⑥	**6,822,590원**	✅
⑧	**1일 평균임금**	⑦ ÷ 92	**74,159원**	✅
⑨	**1일 통상임금**	8h × 10,320	**82,560원**	✅
⑩	**Max 비교**	Max(74,159, 82,560)	**82,560원** (통상임금 선택)	✅
⑪	**퇴직금**	82,560 × 30 × (457÷365)	**3,101,089원**	✅

> ***확인**: 평균임금(74,159) < 통상임금(82,560)이므로 근로기준법 제2조 제2항에 따라 통상임금이 퇴직금 기준 단가로 선택됩니다.*
>
> *※ 연차 15일은 1년 이상 근무 + 80% 이상 출근율 충족을 전제로 한 발생 확정 일수입니다. 실제로는 출근율·회계연도 기준·입사일 기준·비례연차 등 변수에 따라 달라질 수 있습니다.*

A-2. 시나리오 B: 단시간(20h) — 통상임금 비교 발동 시뮬레이션

가상 데이터 설정

항목	값
입사일	2025-01-01
퇴사일	2026-06-30
주간 소정근로	20시간 (단시간)
시급	10,320원

계산 추적

단계	항목	산식	결과
①	재직일수		**546일**
②	역월 3개월	6/30 → 3/30	**92일**
③	hoursMultiplier	(20÷40) × 8	**4.0h**
④	주간 유급시간	20 + 4	**24h**
⑤	3개월 기본+주휴	24 × (92÷7) × 10,320	**3,255,223원**
⑥	연차 산입	15 × 4 × 10,320 × (92÷365)	**156,072원**
⑦	**평균임금**	3,411,295 ÷ 92	**37,079원**
⑧	**통상임금**	4 × 10,320	**41,280원**
⑨	Max 비교	37,079 < 41,280	**41,280원** (통상임금 선택)
⑩	**퇴직금**	41,280 × 30 × (546÷365)	**1,852,511원**

> ***확인**: 단시간 근로자나 근무일 변동이 큰 근로자는 최근 3개월 실제 지급임금이 낮게 형성될 수 있어, 평균임금이 통상임금보다 낮아질 가능성이 있습니다. 비교 결과 통상임금이 선택될 가능성이 높습니다.*

A-3. 시나리오 C: 실제 출근기록 보유 — 실데이터 경로에서도 비교 로직 작동

> ***목적**: A-2(이론 추산)와 동일한 단시간 근로자 조건에서, **실제 출근 기록이 있는 경우**에도 `Max(평균, 통상)` 비교가 정상 작동하는지 확인합니다.*

가상 데이터 설정

항목	값

입사일	2025-01-01
퇴사일	2026-06-15
재직일수	546일 (≥365 ✓)
주간 소정근로	20시간 (주4일 × 하루 5시간)
시급	10,320원
역월 3개월 기간	92일 (3/15~6/15)
실제 출근기록	52일 (주4일 × 13주)

계산 추적

단계	항목	산식	결과
①	실근무 총액	52일 × 5h × 10,320	**2,683,200원**
②	주휴수당 가산	(20÷40)×8 × 10,320 × (92÷7)	**542,537원**
③	연차수당 산입	15일 × 4h × 10,320 × (92÷365)	**156,072원**
④	3개월 총액	① + ② + ③	**3,381,809원**
⑤	**평균임금**	3,381,809 ÷ 92	**36,759원**
⑥	**통상임금**	4h × 10,320	**41,280원**
⑦	Max 비교	36,759 < 41,280	**41,280원** (통상임금 선택)

> ***확인***: *단시간 근로자나 근무일 변동이 큰 근로자는 최근 3개월 실제 지급임금이 낮게 형성될 수 있어, 평균임금이 통상임금보다 낮아질 가능성이 있습니다. A-2(이론 추산)와 동일한 조건에서* ***실데이터 경로로 계산해도*** *비교 로직이 정상 작동하며, 결과값(41,280원)이 일치함을 확인합니다.*

A-4. 시나리오 D: 5인 미만 — 연차수당 미산입 시뮬레이션

구분	연차 발생일수	3개월 연차수당 산입액	차이
5인 이상	15일	**312,145원**	—
5인 미만	0일	**0원**	-312,145원

> ***확인***: *5인 미만 사업장은* `_estimateAnnualLeaveDays()` *가 0을 반환하여 연차수당이 평균임금에 산입되지 않습니다. 이는 연차 법적 의무가 없는 5인 미만 사업장에 대한 적법한 처리입니다.*

A-5. 시나리오 E: 미사용 연차수당 정산 — AnnualLeaveCalculator 연동 검증

> ***검증 목적***: `SeveranceCalculator` *의* `remainingLeaveDays` *와* `annualLeavePayout` *이* `AnnualLeaveCalculator.calculateAnnualLeaveSummary()` *와 일치하는지 교차 검증합니다.*

가상 데이터: 입사 2025-01-01, 퇴사 2026-06-15, 전일제(40h), 시급 10,320원

#	서브 케이스	검증 내용	결과
E-1	사용 0일	`SeveranceCalc.remainingLeaveDays = AnnualLeaveCalc.remaining`	✅
		`SeveranceCalc.annualLeavePayout = AnnualLeaveCalc.annualLeaveAllowancePay`	✅
		잔여일 > 0, 수당 > 0	✅
E-2	5일 사용	잔여일 = E-1 잔여 - 5 (5일 감소)	✅

		수당 차이 = 5 × 8h × 10,320 = **412,800원**	✅
E-3	5인 미만	잔여 = 0, 수당 = 0	✅
E-4	단시간(20h)	수당 < 전일제 수당 (비례 축소)	✅

> ***확인***: `SeveranceCalculator` *는 연차수당을 직접 계산하지 않고,* `AnnualLeaveCalculator` *에 위임하여 결과를 그대로 반환합니다 (severance_calculator.dart L42-60). 따라서 두 엔진 간* ***연차 발생·사용·잔여·수당 산출이 완전히 동기화****되어 있습니다.*

A-6. 퇴직금 정산 엣지 케이스 시뮬레이션 결과

> ***검증 목적***: `SeveranceCalculator` *엔진이 가상 데이터를 기반으로 다양한 엣지 케이스에서 노동법 기준을 반영하도록 설계되었는지 수치로 시뮬레이션합니다. (단일 시급 10,000원 가정)*

케이스 구분	가상 조건	산정된 평균 일급	최종 퇴직금	근로기준법 판정
Case 1 (딱 1년 만근)	365일 근무, 주 15시간	15,000원	**900,000원**	✅ **대상 (지급)** (15,000원 × 30일 × 1.0년)
Case 2 (하루 부족)	364일 근무, 주 15시간	-	**0원**	❌ **미대상** (365일 미만 기각)
Case 3 (초단시간)	730일 근무, 주 14시간	-	**0원**	❌ **미대상** (4주 평균 15시간 미만 가정)
Case 4 (통상임금 하한선)	1,095일 근무, 주 40시간 (5인 미만)	**80,000원**	**7,200,000원**	✅ **대상 (하한선 방어)** (산정 평균임금이 통상임금(1일 소정근로시간 × 시급)보다 낮은 경우, 근로기준법 제2조 제2항에 따라 통상임금을 기준으로 보정)
Case 5 (장기 근속 4년)	1,461일 근무, 주 40시간 (5인 이상)	**80,000원** 이상	**9,606,575원**	✅ **대상 (일할 계산)** (4년 + 윤년 1일 치가 정확하게 소수점 단위로 합산됨)

> ***검증 결과***: *엔진은 재직일수, 소정근로시간, 평균임금·통상임금 비교 규칙 등 근로기준법 기준을 반영하여 계산하도록 설계되었으며, 평균임금이 낮게 산정되는 경우 통상임금 하한선 비교 로직을 통해 과소지급 가능성을 방어합니다. 또한, 윤년을 포함한 재직일수 기반 일할 계산이 정상적으로 반영됨을 확인했습니다.*

부록 검증 결과: 6개 시나리오 모두 수기 계산과 엔진 출력이 오차 범위 내에서 일치합니다. (전체 테스트: 114개 통과)

부록 B. 알려진 한계 및 향후 과제

B-1. 사업장 규모(5인 이상/미만) 전환 시 퇴직금 산정 한계

현재 동작: `SeveranceCalculator` 는 호출 시점의 `isFiveOrMore` 단일 값만 사용합니다.

```
// severance_calculator.dart L46
isFiveOrMore: isFiveOrMore,  // ← 퇴직 시점의 단일 값
```

문제 시나리오:

시점	사업장 규모	연차 발생	현재 엔진 처리
입사 ~ 2년 근무	5인 이상	매년 15일 발생	—
퇴사 직전	**5인 미만으로 축소**	신규 발생 중단	—
퇴직금 산정	`isFiveOrMore = false`	—	연차 산입 **0일** (과소 산정 위험)

법적 해석: 재직 중 5인 이상일 때 발생한 연차는 이후 규모 축소로 소멸하지 않습니다. 그러나 현재 엔진은 이 전환 이력을 추적하지 않아, 퇴직 시점이 5인 미만이면 **과거에 발생한 연차까지 전부 0으로 처리**됩니다.

영향 범위:

- 퇴직금 평균임금의 연차수당 산입액
- 미사용 연차수당 정산액 (`annualLeavePayout`)

구현 난이도 평가: 중 (아래 B-3 참조)

B-2. 사용촉진 면제액의 퇴직정산 표시

현재 동작: `AnnualLeaveCalculator` 는 사용촉진 2차 완료 시 `promotionExemptPayoutAmount` 를 정확히 산출합니다.

항목	위치	퇴직정산 연동
`remaining` (잔여일)	`AnnualLeaveSummary`	✅ `ExitSettlementResult.remainingLeaveDays`에 반영
`annualLeaveAllowancePay` (수당)	`AnnualLeaveSummary`	✅ `ExitSettlementResult.annualLeavePayout`에 반영
`promotionExemptPayoutAmount` (면제액)	`AnnualLeaveSummary`	❌ `ExitSettlementResult`에 **미포함**

실질적 영향: 촉진 완료로 소멸된 연차는 `remaining` 에서 이미 제외되므로 **수당 산출 자체는 정확**합니다. 다만 퇴직정산서(PDF)에 "사용촉진 완료로 N일분 수당 면제" 안내가 표시되지 않아 사장님 입장에서 금액 근거가 불투명할 수 있습니다.

구현 난이도: 하 — `ExitSettlementResult` 에 필드 1개 추가 + UI 표시

B-3. 5인 전환 로직 구현 난이도 상세

항목	평가	설명
데이터 모델	하	Worker에 `fiveOrMoreHistory: [{date, value}]` 배열 추가
Firestore 마이그레이션	하	기존 `isFiveOrMore` bool → 배열 전환, 기존 값을 초기 항목으로 보존
AnnualLeaveCalculator 수정	**중**	배치별 발생 시점에 해당 기간의 규모를 조회해야 함 — 현재 단일 bool 가정이 여러 곳에 산재
SeveranceCalculator 수정	하	`_estimateAnnualLeaveDays()`에 이력 전달만 하면 됨
UI (설정 화면)	중	전환 날짜 입력 UX 필요 — 사장님이 "언제부터 5인 미만이 되었는가" 입력
테스트	중	전환 전후 경계 케이스 다수

> ***총평**: 핵심 난이도는 `AnnualLeaveCalculator` 의 배치별 규모 조회와 사장님 UX 설계입니다. 엔진 자체보다 **"사장님이 전환 시점을 명확히 인지하고 있는가?"** 라는 UX 문제가 더 큽니다. 실무적으로는 **수동 보정(`annualLeaveManualAdjustment`)으로 대응 가능**하므로, 우선순위는 높지 않습니다.*

부록 C. 자동화 테스트 전체 목록 (113개) — 엔진 계산 로직 검증

> ***목적**: 본 문서(§1~§8)에서 설명한 모든 계산 로직이 가상 데이터 기반 단위 테스트로 시뮬레이션되었음을 확인합니다. 각 테스트는 `PayrollCalculator.calculate()` 엔진을 직접 호출하여, 입력 데이터(출퇴근 기록·근로자 설정)에 대한 출력 값(급여·수당)이 기대치와 오차 범위 내에서 일치하는지 검증합니다.*

실행 방법

```
# 전체 테스트 일괄 실행 (113개)
cd packages/shared_logic
flutter test
```

```
# 개별 파일 실행 (상세 출력)
flutter test test/compliance/<파일명>.dart --reporter=expanded
```

C-1. 출퇴근 보정 및 기본급 (§2, §3, §7.4.3)

테스트 파일: `test/compliance/report_proof_clockin_test.dart` (18개)

#	매뉴얼	테스트명	검증 내용	기대값
1	§2.1	조기 출근 Cap: 8:40 출근 → 정시 9:00 보정	`payrollEffectiveClockIn` 함수가 스케줄 이전 출근을 정시로 보정	`effective == 09:00`
2	§2.1	지각 유예 5분: 9:03 출근 → 9:00 인정	`graceMinutes=5` 이내 지각 → 정시 인정	`effective == 09:00`
3	§2.1	지각 유예 초과 8분: 9:08 출근 → 9:08 그대로	`graceMinutes=5` 초과 지각 → 실제 시각 적용	`effective == 09:08`
4	§2.1	연장 미승인: 18:20 퇴근 → 18:00 절삭	`overtimeApproved=false` → 정시 Cap 적용	`effective == 18:00`
5	§2.1	연장 승인: 18:20 퇴근 → 18:20 그대로	`overtimeApproved=true` → 실제 퇴근 시각 인정	`effective == 18:20`
6	§2.3	8시간 이상 → 최소 60분 휴게 강제	9h 체류 시 사장님 미설정이어도 법정 60분 차감	`appliedBreak ≥ 60`
7	§2.3	4~8시간 → 최소 30분 휴게 강제	5h 체류 시 법정 30분 차감	`appliedBreak ≥ 30`
8	§2.4	Unplanned → 급여 제외	스케줄 외 무단 출근 → 정산 제외	`isIncluded == false`
9	§2.4	pending_approval → 급여 제외	승인 대기 → 정산 제외	`isIncluded == false`
10	§2.4	pending_overtime → 급여 제외	연장 승인 대기 → 정산 제외	`isIncluded == false`
11	§2.4	early_leave_pending → 급여 제외	조퇴 승인 대기 → 정산 제외	`isIncluded == false`
12	§2.4	Normal → 급여 포함	정상 출근 → 정산 포함	`isIncluded == true`
13	§2.4	UnplannedApproved → 급여 포함	승인된 비예정 출근 → 정산 포함	`isIncluded == true`
14	§2.4	early_clock_out → 급여 포함	승인된 조퇴 → 정산 포함	`isIncluded == true`
15	§3.1	분 단위 정밀 계산: 7시간 30분 × 10,320	7.5h × 10,320 = 77,400원	`basePay ≈ 77,400`
16	§3.1	유급 휴게: breakPay > 0	`isPaidBreak=true` → 1h × 10,320 = 10,320원	`breakPay ≈ 10,320`
17	§3.3	수습 기간 시급 = 원래 시급 × 0.9	`isProbation=true` → floor(10,320×0.9) = 9,288원 기준	`basePay ≈ 8h × 9,288`
18	§7.4.3	월 중간 시급 변경 → 일자별 분할 적용	3일@10,320 + 2일@11,000 = 423,680원	`basePay ≈ 423,680`

C-2. 가산수당·근로자의 날·월급제 (§3.2, §4, §5.3)

테스트 파일: `test/compliance/report_proof_premium_test.dart` (7개)

#	매뉴얼	테스트명	검증 내용	기대값
1	§4.1	5인 이상: 연장+야간 가산 → premiumPay > 0	14~24시 근무(연장 1h+야간 2h) → 0.5배 가산	`premiumPay > 0`
2	§4.1	5인 미만: 가산 0원	동일 근무, `isFiveOrMore=false`	`premiumPay == 0`
3	§4.1	5인 이상 vs 5인 미만 → 가산수당 차이	동일 입력, 5인 분기만 변경 → 총급여 차이 존재	`totalPay(5+) > totalPay(5-)`
4	§5.3	시급제 주 40h: 미출근 시 8h × 시급 유급	5/1 미출근이어도 `laborDayAllowancePay` 발생	`≈ 82,560원`
5	§5.3	초단시간 주 14h: 비례 유급	(14/40)×8×10,320 = 28,896원	`≈ 28,896원`
6	§3.2	중도 입사 → proRataRatio < 1.0	7/16 입사, 7월 정산 → 일할 계산	`proRataRatio < 1.0`
7	§2.4	스케줄 없는 출근 → 보정 없이 실시간 인정	`scheduledShiftStartIso=null` → Cap 미적용	`basePay ≈ 7.5h × 10,320`

C-3. 주휴수당 (§5.1)

테스트 파일: `test/compliance/weekly_holiday_pay_test.dart` (7개)

#	매뉴얼	테스트명	검증 내용	기대값
1	§5.1	주 40시간 만근 → 주휴수당 = 8h × 시급	5일 만근 → `weeklyHolidayPay` 정상 지급	`≈ 82,560원`
2	§5.1	주 15시간 미만 → 주휴수당 대상 아님	초단시간 → 주휴수당 0원	`== 0`
3	§5.1	주 15시간 경계값 → 주휴수당 대상	경계값 검증	`> 0`
4	§5.1	만근 미달(결근) 시 주휴수당 미지급	`weeklyHolidayBlockedByAbsence = true`	`== 0`
5	§5.1	주 20시간 파트타임 → 비례 주휴수당	(20/40)×8 = 4h 분의 주휴수당	`≈ 4h × 시급`
6	§5.1	조퇴 → 출근했으므로 주휴수당 유지	수요일 3시간만 근무해도 주휴 유지	`> 0`
7	§5.1	만근 vs 결근 → 급여 차이 = 일급 + 주휴수당	결근 시 일급+주휴 이중 손실	차이 `= 165,120원`

C-4. 가산수당 상세 (§4)

테스트 파일: `test/compliance/premium_pay_test.dart` (5개)

#	매뉴얼	테스트명	검증 내용	기대값
1	§4	정상 8시간 근무 → 연장수당 0원	8h 이내 → 가산 없음	`premiumPay == 0`
2	§4.1	5인 이상: 10시간(2시간 연장) → 1.5배	연장 2h × 0.5배 가산	`premiumPay > 0`
3	§4.1	5인 미만: 10시간(2시간 연장) → 가산 0원	가산수당 의무 없음	`premiumPay == 0`
4	§4.1	5인 이상: 야간근로(22~06시) → 0.5배 가산	야간 시간대 → 가산 발생	`premiumPay > 0`
5	§4.1	5인 이상: 토요일(휴무일) 대타 → 휴일 가산	휴일근로 가산	`premiumPay > 0`

C-5. 연차 휴가 (§6.1)

테스트 파일: `test/compliance/annual_leave_compliance_test.dart` (8개)

#	매뉴얼	테스트명	검증 내용
1	§6.1	1년 미만: 5개월 만근 → 5일 발생	월별 1일씩
2	§6.1	1년 미만: 최대 11일까지만 발생	상한 검증
3	§6.1	1년 차: 11일 + 15일 = 26일	누적 검증
4	§6.1	장기근속: 3년차 16일, 상한 25일	가산 검증
5	§6.1	5인 미만: 연차 0일	법적 의무 없음
6	§6.1	초단시간(주 14h): 연차 미발생	15h 미만 제외
7	§6.1	퇴사 시 잔여 연차 → 수당 정산	잔여일 × 8h × 시급
8	§6.1	단시간(주 20h): 비례 연차수당	Double Discount 방지

테스트 파일: `test/compliance/annual_leave_usage_test.dart` (8개)

#	매뉴얼	테스트명	검증 내용
1	§6.1.2	연차 사용일 유급 보장	`basePay` 제외 + `annualLeaveUsedPay`로 유급 지급
2	§6.1.2	연차 사용일 = 출근 간주 → 주휴수당 유지	`isAttendanceEquivalent=true`
3	§6.1.2	연차 없이 결근 → 주휴수당 0원 (대조군)	결근 vs 연차 비교
4	§6.1	잔여일수: 26일 - 3일 = 23일	사용 차감 정확성
5	§6.1	퇴사 정산: 사용 3일 반영 후 잔여분 수당	수당 정확성
6	§6.1	취소 복원: 3일 → 1일 취소 → used 2일	원자적 롤백
7	§6.1	음수 차감 방지: used 0 → 취소 → 0 유지	`clamp(0, ∞)`
8	§6.1.2	월급제 중복 방지	`annualLeaveUsedPay`가 `finalTotalPay`에 미포함

C-6. 연차 사용촉진 (§6.1.1)

테스트 파일: `test/compliance/leave_promotion_test.dart` (10개)

#	매뉴얼	테스트명	검증 내용
1	§6.1.1	정기 연차: 1차 촉진 기한 = 소멸 6개월 전	`firstNoticeDeadline` 정확성
2	§6.1.1	정기 연차: 2차 촉진 기한 = 소멸 2개월 전	`secondNoticeDeadline` 정확성
3	§6.1.1	1년 미만: 1차 촉진 = 소멸 3개월 전	단축 기한 검증
4	§6.1.1	1년 미만: 2차 촉진 = 소멸 1개월 전	단축 기한 검증
5	§6.1.1	촉진 완료 → isExemptFromPayout = true	수당 면제 플래그
6	§6.1.1	촉진 미완료 → isExemptFromPayout = false	수당 지급 대상
7	§6.1.1	촉진 만료 → isExemptFromPayout = false	기한 초과 시 수당 지급
8	§6.1.1	연차 유형 라벨: 정기 vs 1년 미만 구분	자동 분기
9	§6.1.1	촉진 윈도우 진입 → promotionStatuses 자동 생성	엔진 자동 감시
10	§6.1.1	촉진 완료 로그 → promotionExemptPayoutAmount 반영	면제액 산출

C-7. 퇴직금 (§6.2)

테스트 파일: `test/compliance/severance_pay_test.dart` (8개)

#	매뉴얼	테스트명	검증 내용
1	§6.2.1	초단시간(주 14h) 400일 → 퇴직금 0원	자격 배제
2	§6.2.1	주 15시간 정확히 + 1년 이상 → 자격 있음	경계값
3	§6.2.2	5월 31일 퇴사 → 역월 92일	90일 아님
4	§6.2.3	5인 미만 → 연차수당 산입 0원	법적 의무 없음
5	§6.2.1	입사 6개월 → 퇴직금 0원	1년 미만
6	§6.2.1	365일 경계값 → 자격 있음	경계값
7	§6.2.4	단시간(주 20h) → 통상임금 비례 축소	Max 비교 발동
8	§6.2	지급기한 = 퇴사일 + 14일	법정 기한

테스트 파일: `test/compliance/severance_pay_proof_test.dart` (5개)

#	매뉴얼	테스트명	검증 내용
A	부록A-1	전일제 이론 추산: 10단계 전수 검증	평균임금 vs 통상임금 비교
B	부록A-2	단시간(20h): 통상임금 비교 발동	평균임금 < 통상임금 비교
C	부록A-3	실데이터 60일: 실데이터 경로 검증	출근기록 기반 산출
D	부록A-4	5인 미만: 연차수당 산입 = 0	총액 차이 확인
E	부록A-5	미사용 연차수당: 엔진 연동 검증	SeveranceCalc ↔ AnnualLeaveCalc 동기화

C-8. 5인 이상/미만 판정 (§1)

테스트 파일: `test/compliance/five_or_more_test.dart` (11개)

#	매뉴얼	테스트명	검증 내용
1	§1.1	단순 평균 5인 이상 → 5인 이상	기본 판정
2	§1.1	단순 평균 5인 미만 → 5인 미만	기본 판정
3	§1.1	★ 평균 5인 미만이지만 과반수 예외 → 5인 이상	1/2 예외 조항
4	§1.1	★ 평균 5인 이상이지만 역방향 예외 → 5인 미만	역방향 1/2 예외
5	§1.3	경계값: 정확히 절반 → 5인 이상	경계값
6	§1.3	경계값: 과반수 1일 미달 → 5인 미만	경계값
7	§1.1	가동일 0일 → 5인 미만	예외 처리
8	§1.1	매일 정확히 5명 → 5인 이상	정확 경계
9	§1.3	★ 데모 재현: 8명 교대근무 → 예외 적용	실사례 검증
10	§1.4	1개월 장기: 과반수 예외 장기 작동	장기 검증
11	§1.2	스냅샷 고정 원칙	급여 확정 후 불변

C-9. 최저임금 (§3.1) 및 월급제 포괄임금 (§3.2)

테스트 파일: `test/compliance/minimum_wage_test.dart` (5개)

#	매뉴얼	테스트명	검증 내용
1	§3.1	2026년 최저시급 = 10,320원	상수 검증
2	§3.1	월급제 기본급 → 최저임금 미달 시 Hard Block	`minimumWageHardBlock=true`
3	§3.1	시급제 → 최저임금 소급 적용	자동 상향
4	§3.1	비과세 식대 한도 = 200,000원	상수 검증
5	§3.1	기본급 = 최저시급 × 209h	역산 검증

테스트 파일: `test/compliance/monthly_inclusive_wage_test.dart` (14개)

#	매뉴얼	테스트명	검증 내용
1	§3.2	기본급 + 식대 + 고정OT = 목표 총급여	250만원 역산
2	§3.2	★ 만근 시 엔진 계산 = 설정된 목표 급여	오차율 0%
3	§3.2	근로자의 날 출근 시 가산수당 발생	휴일 가산
4	§3.2	통상시급에서 고정연장수당 제외	대법원 판례
5	§3.2	최저임금 미달 시 Hard Block	이중 락
6	§3.2	5인 미만: 연장 가산율 1.0배	가산 없음
7	§3.2	비과세 식대 200,000원 처리	세금 계산
8~14	§3.2	고정OT 초과분 보전, 일할계산, 중도입사 등	포괄임금 엣지 케이스

C-10. 검증 총괄

```
┌──────────────────────────────────────────────────────────┐
│  전체 테스트: 113개 — ALL PASSED ✅                        │
│                                                            │
│  테스트 파일: 12개                                         │
│  커버 매뉴얼 섹션: §1 ~ §8 (부록 A·B 포함)                 │
│  실행 명령: flutter test (packages/shared_logic)           │
│  실행 시간: ~2초                                           │
│                                                            │
│  ⚠️ 테스트 범위 외 (Integration Test 필요):                │
│    - 바코드/WiFi 출퇴근 하드웨어                           │
│    - GPS 반경 검증                                         │
│    - Firestore 읽기/쓰기                                   │
│    - PDF 렌더링 (Golden Test 권장)                         │
│    - FCM 푸시 알림                                         │
└──────────────────────────────────────────────────────────┘
```

> ***최종 검증일**: 2026-05-10 **검증 환경**: Flutter 3.x, Dart 3.x, macOS **검증 결과**: 113개 테스트 전부 통과. 모든 계산 경로에서 수기 계산과 엔진 출력이 오차 범위 내에서 일치합니다.*